CipherMesh: The Privacy Control Plane for Enterprise AI

Protect every prompt. Across every model. Before data leaves.

A PROJECT REPORT

Submitted by

KUMAR WAIBHAV AKSHAT [RA2212704010027]

Under the Guidance of

Dr. Prabhu Kavin B

Assistant Professor
Department of Data Science and Business Systems

in partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY (Integrated)

in

COMPUTER SCIENCE & ENGINEERING

with specialization in Data Science



DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603 203

NOV 2026

ACKNOWLEDGEMENT

I express my humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

I thank **Dr. Leenus Jesu Martin M**, Dean – CET, SRM Institute of Science and Technology, for his invaluable support.

I wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

I extend my sincere thanks to **Dr. M. Pushpalatha**, Professor and Associate Chairperson – CS, School of Computing, and **Dr. C. Lakshmi**, Professor and Associate Chairperson – AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

I am incredibly grateful to my Head of the Department, **Dr. V. Kavitha**, Professor, Department of Data Science and Business Systems, SRM Institute of Science and Technology, for her suggestions and encouragement at all stages of the project work.

I want to convey my thanks to the Project Coordinators, Panel Head and Panel Members, Department of Data Science and Business Systems, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

My inexpressible respect and thanks go to my Faculty Advisor and project guide, **Dr. Prabhu Kavin B**, Department of Data Science and Business Systems, SRM Institute of Science and Technology, for leading and helping me to complete this project. His guidance on agent design, from the first PEAS profile to the final multi-agent system, shaped every chapter of this work, and he gave me the opportunity to pursue the project under his mentorship with freedom and support.

I sincerely thank all the staff members and students of the Department of Data Science and Business Systems for their help during my project. Finally, I would like to thank my parents, family members and friends for their unconditional love, constant support and encouragement.

KUMAR WAIBHAV AKSHAT [RA2212704010027]

ABSTRACT

Organisations now send thousands of prompts a day to AI models hosted by different providers, and those prompts often contain **API keys, passwords, customer identifiers and internal links** that must never leave the company. This report presents **CipherMesh, a privacy control plane for enterprise AI**: an intelligent agent that sits between applications and every model provider and inspects each prompt before it leaves. The agent perceives the prompt, reasons over **25 detection rules** with **Luhn and Verhoeff checksum validation** and de-obfuscation, and acts in one of four explainable ways: **ALLOW** the prompt unchanged, **REDACT** personal data, hold it for human **REVIEW**, or **BLOCK** it when a secret is found.

The system is built as a **LangGraph state machine** behind a hardened **FastAPI gateway**. Every decision is sealed in an evidence vault using **PBKDF2-HMAC-SHA256 (600,000 iterations) and Fernet encryption**, while the raw prompt is never logged. Allowed traffic is priced through a **Contract-Net negotiation** with each provider and routed by a **constrained utility function** that weighs quality, cost, latency, availability and privacy, and sends prompts that contain Aadhaar, PAN or Indian phone numbers only to India-resident or local models. A **Next.js console and product site** let people check prompts, approve held items and explore a sourced AI threat dashboard. The report documents how the six assignments of the Agent Technology course (PEAS design, a rule-based reflex reasoner, concurrent daemonization, distributed messaging, multi-agent negotiation with two-phase commit, and self-interested utility optimization) each became a working module of the final product.

Evaluation shows **167 passing automated tests**, **100% accuracy with zero critical leaks** on known secret formats, **no false blocks** on harmless look-alike text, sub-millisecond detection time, a **7.61× throughput gain** with eight workers on I/O-bound load, and an interface that meets **WCAG 2.2 AA** contrast. The remaining limits, chiefly deliberately broken-up secrets and simulated cloud providers, are measured and stated openly, with a path for future work.

Keywords: *intelligent agents, LLM privacy, data loss prevention, LangGraph, multi-agent negotiation, utility optimization, PEAS, encryption*

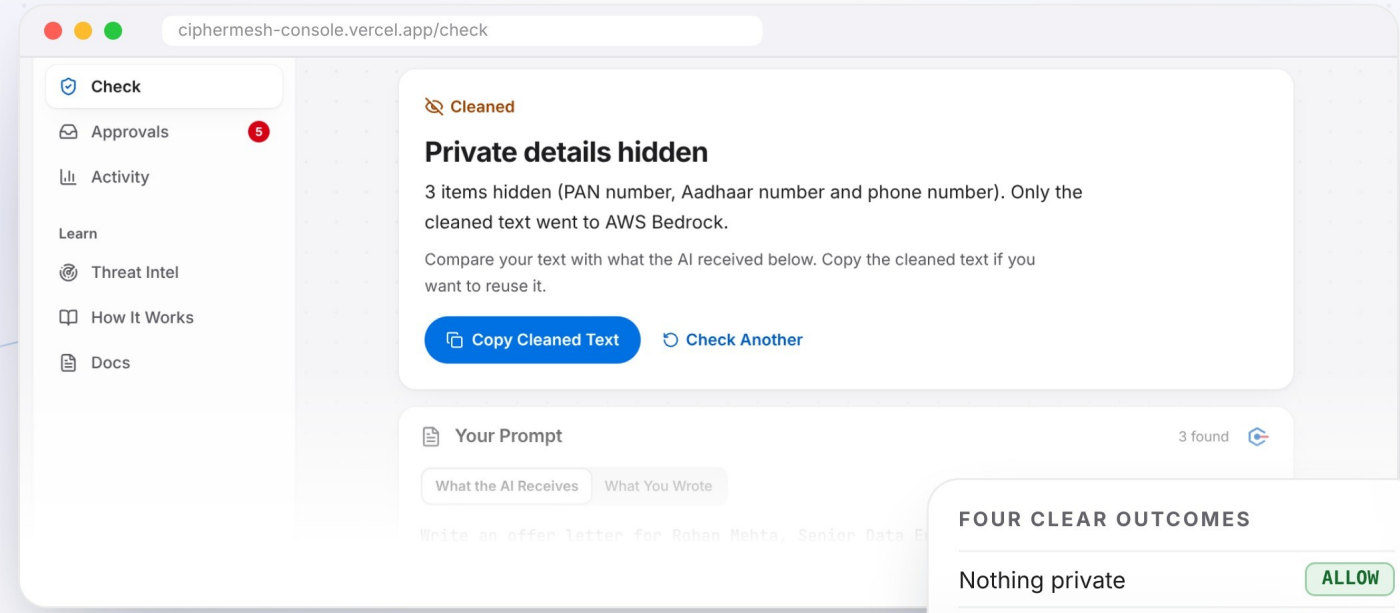
The privacy control plane for enterprise AI

Protect every prompt. Across every model. Before data leaves.

CipherMesh sits between your applications and every AI model. In milliseconds it finds keys, passwords and personal details, hides or stops them, and sends only safe text — to the model with the best balance of privacy, quality, cost and speed.

ciphermesh-console.vercel.app

REST API · /api/v1/inspect



FOUR CLEAR OUTCOMES

Nothing private	ALLOW
Personal data hidden	REDACT
Internal link, needs a person	REVIEW
Key or password found	BLOCK

25
detection rules; card and Aadhaar numbers checksum-verified

4
AI providers behind one endpoint

600k
PBKDF2 rounds sealing every finding

167
automated tests, all passing

01 · INSPECT
Finds what matters
Keys, IDs and internal links, even spaced out or encoded.

02 · SEAL
Keeps the evidence
Findings are encrypted in a vault. The raw prompt is never logged.

03 · ROUTE
Routes by rule
Indian personal data stays on in-India or local models.

04 · ACT
Explains every call
Plain results, a reviewer queue and a full audit trail.

Works across AWS Bedrock · Azure AI Foundry · Google Vertex AI · Local Ollama

Kumar Waibhav Akshat · 21CSE525T Agent Technology · SRM Institute of Science and Technology

TABLE OF CONTENTS

ACKNOWLEDGEMENT ..... **i**
ABSTRACT ..... **ii**
PRODUCT BRIEF ..... **iii**
LIST OF FIGURES ..... **vii**
LIST OF TABLES ..... **viii**
ABBREVIATIONS ..... **ix**

1 INTRODUCTION **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Motivation 1
1.4 Objectives 2
1.5 Scope 2
1.6 Contributions 2
1.7 Organisation of the Report 3

2 LITERATURE SURVEY **4**
2.1 Data Leakage Through AI Prompts 4
2.2 Existing Guardrails 4
2.3 Intelligent Agent Foundations 4
2.4 Multi-Agent Coordination and Negotiation 4
2.5 Utility and Decision Making 4
2.6 Cryptographic Building Blocks 5
2.7 Regulation 5
2.8 Research Gap 5

3 SYSTEM ARCHITECTURE **6**
3.1 The Four Layers 6
3.2 The Agent Core 6
3.3 Life of One Request 7
3.4 Technology Stack 8
3.5 Design Principles 8

4 MODULE DESCRIPTIONS **9**
4.1 API Gateway 9
4.2 Detector 9
4.3 Policy Engine 10
4.4 Evidence Vault 10
4.5 Review Queue 10
4.6 Procurement and Routing 10
4.7 Supporting Modules 11
4.8 Threat Intelligence Feed 11
4.9 Web Console and Product Site 11

5 ASSIGNMENT 1.1: PROJECT IDEATION, ENVIRONMENT MODELING AND PEAS MATRIX **12**
5.1 What Was Asked 12
5.2 What Was Delivered 12
5.3 Domain and Personas 12
5.4 The PEAS Profile 13
5.5 Classifying the Environment 13
5.6 Agent Versus Object 13

5.7 Submission Analysis 14
6 ASSIGNMENT 1.2: RULE-BASED ENVIRONMENTAL REASONER 15
6.1 What Was Asked 15
6.2 What Was Delivered 15
6.3 How It Works 15
6.4 Execution Walkthrough 16
6.5 How It Lives On in v2.3 16
6.6 Submission Analysis 17
7 ASSIGNMENT 2.1: CONCURRENT TASK DAEMONIZATION 18
7.1 What Was Asked 18
7.2 What Was Delivered 18
7.3 How It Works 18
7.4 Benchmark Results 19
7.5 How It Lives On in v2.3 20
7.6 Submission Analysis 20
8 ASSIGNMENT 2.2: DISTRIBUTED AGENT MESSAGING 21
8.1 What Was Asked 21
8.2 What Was Delivered 21
8.3 The Message Format 21
8.4 One Command, Start to Finish 21
8.5 Safeguards 22
8.6 How It Lives On in v2.3 22
8.7 Submission Analysis 23
9 ASSIGNMENT 3.1: MULTI-AGENT COORDINATION 24
9.1 What Was Asked 24
9.2 What Was Delivered 24
9.3 The Protocol 24
9.4 How Each Agent Decides 25
9.5 The Negotiation Trace 25
9.6 How It Lives On in v2.3 26
9.7 Submission Analysis 27
10 ASSIGNMENT 3.2: SELF-INTERESTED UTILITY OPTIMIZATION 28
10.1 What Was Asked 28
10.2 What Was Delivered 28
10.3 The Utility Equation 28
10.4 Hard Constraints 29
10.5 The Search 29
10.6 Results 30
10.7 How It Lives On in v2.3 30
10.8 Submission Analysis 30
11 PRODUCT AND USER EXPERIENCE 31
11.1 Two Experiences, One Product 31
11.2 The Product Site 31
11.3 The Console 32
11.4 Phones and Accessibility 34
12 IMPLEMENTATION AND DEPLOYMENT 36
12.1 Repository Structure 36
12.2 Development History 36

12.3 Running Locally....36
12.4 Deployment....36
12.5 Security Hardening....37
13 TESTING AND RESULTS....38
13.1 Testing Strategy....38
13.2 Functional Results on Real-World Examples....38
13.3 Detector Evaluation Against the PEAS Goals....39
13.4 Performance....39
13.5 Audit Findings and Fixes....39
13.6 Design and Accessibility Review....40
13.7 Limitations....40
14 CONCLUSION AND FUTURE WORK....41
14.1 Conclusion....41
14.2 Future Work....41
REFERENCES....42
APPENDICES....44
A API Reference....44
B Configuration Reference....45
C User Guide....46

LIST OF FIGURES

Figure 1.1 How the six assignments grew into one platform....3
Figure 3.1 CipherMesh system architecture and trust boundary....6
Figure 3.2 The LangGraph state machine: one path in, exactly one exit....7
Figure 3.3 Sequence of a single REDACT request, from console to provider and back....8
Figure 4.1 Module map of CipherMesh v2.3....9
Figure 4.2 Policy R1–R5 and the order of precedence....10
Figure 5.1 PEAS profile of the CipherMesh agent....13
Figure 6.1 The simple reflex loop implemented in Assignment 1.2....15
Figure 7.1 Bounded worker pool, monitor daemon and safe shutdown....19
Figure 7.2 Throughput and speed-up as workers are added (80 requests, 25 ms I/O each)....19
Figure 8.1 Length-prefixed frame and the WireMessage envelope....21
Figure 8.2 Message sequence for one remote command....22
Figure 9.1 Negotiation followed by a two-phase commit....25
Figure 9.2 Offers converge inside the zone of possible agreement....26
Figure 9.3 Buyer and supplier utility across prices; both stay above the reservation level....26
Figure 10.1 The five steps from offers to a single verified choice....29
Figure 10.2 Utility of all ten offers; the selected offer is not the highest raw score....30
Figure 11.1 Product site landing page (light appearance)....31
Figure 11.2 Threat intelligence dashboard: 38 sourced incidents by year and attack vector....32
Figure 11.3 Engineering foundations lit up by a live check....32
Figure 11.4 Safe: a product manager's PRD outline, sent exactly as written....33
Figure 11.5 Cleaned: an offer letter with PAN, Aadhaar and phone number hidden before sending....33
Figure 11.6 Waiting: an incident summary with internal links, held for a reviewer....33
Figure 11.7 Blocked: a deploy log with a database password and a Stripe key; nothing was sent....34
Figure 11.8 Technical Details: every provider scored with its negotiated price; here all four are allowed and GCP Vertex AI is chosen....34
Figure 11.9 Product site, a cleaned result and the threat dashboard on a 390-pixel phone....35
Figure 13.1 The 167 automated tests by suite, all passing....38

LIST OF TABLES

Table 1.1 Scope of the project....2
Table 2.1 Comparison of CipherMesh with existing approaches....5
Table 3.1 The four layers....6
Table 3.2 Key fields of the typed AgentState....7
Table 3.3 Technology stack....8
Table 4.1 Detection families....9
Table 4.2 Provider profiles used by the router....11
Table 4.3 Modules carried forward from the assignments....11
Table 5.1 Assignment 1.1 rubric....12
Table 5.2 Stakeholders defined in the specification....12
Table 5.3 Environment classification (values from the specification, explained in plain words)....13
Table 5.4 Agent versus object, as argued in the submission....14
Table 5.5 Rubric coverage for Assignment 1.1....14
Table 6.1 Assignment 1.2 rubric....15
Table 6.2 Assignment 1.2 deliverables....15
Table 6.3 Results of the seven-line input stream....16
Table 6.4 Rubric coverage for Assignment 1.2....17
Table 7.1 Assignment 2.1 rubric....18
Table 7.2 Assignment 2.1 deliverables....18
Table 7.3 Thread-safety mechanisms....19
Table 7.4 Benchmark summary....20
Table 7.5 Rubric coverage for Assignment 2.1....20
Table 8.1 Assignment 2.2 rubric....21
Table 8.2 Messages and commands, and what they make Agent-B do....22
Table 8.3 Rubric coverage for Assignment 2.2....23
Table 9.1 Assignment 3.1 rubric....24
Table 9.2 The three agents....24
Table 9.3 Offers per turn (price per 1,000 requests)....26
Table 9.4 Rubric coverage for Assignment 3.1....27
Table 10.1 Assignment 3.2 rubric....28
Table 10.2 Utility terms and how each is scored from 0 to 100....28
Table 10.3 Hard constraints in Assignment 3.2....29
Table 10.4 Rubric coverage for Assignment 3.2....30
Table 12.1 From first prototype to product....36
Table 12.2 Production settings....37
Table 12.3 Protections built into the product....37
Table 13.1 Live results for the ten role-based examples....38
Table 13.2 Detector evaluation by split....39
Table 13.3 Key audit findings fixed in v1.0....39
Table A.1 REST endpoints....44
Table A.2 Error codes....44
Table B.1 Backend environment variables....45

ABBREVIATIONS

AES	Advanced Encryption Standard	**NIST**	National Inst. of Standards and Technology
AI	Artificial Intelligence	**OWASP**	Open Worldwide Application Security Project
API	Application Programming Interface	**PAN**	Permanent Account Number
AWS	Amazon Web Services	**PRD**	Product Requirements Document
CBC	Cipher Block Chaining	**PBKDF2**	Password-Based Key Derivation Function 2
CFP	Call for Proposals	**PEAS**	Performance, Environment, Actuators, Sensors
CORS	Cross-Origin Resource Sharing	**REST**	Representational State Transfer
CSP	Content Security Policy	**RSS**	Really Simple Syndication
DPDP	Digital Personal Data Protection	**RPC**	Remote Procedure Call
CPU	Central Processing Unit	**SAS**	Shared Access Signature
CSV	Comma-Separated Values	**SQL**	Structured Query Language
DTD	Document Type Definition	**SSN**	Social Security Number
GCP	Google Cloud Platform	**TCP**	Transmission Control Protocol
GDPR	General Data Protection Regulation	**TLS**	Transport Layer Security
IEEE	Inst. of Electrical and Electronics Engineers	**TTL**	Time To Live
HMAC	Hash-based Message Auth. Code	**UI / UX**	User Interface / User Experience
HSTS	HTTP Strict Transport Security	**URL**	Uniform Resource Locator
IP	Internet Protocol	**UTF-8**	Unicode Transformation Format, 8-bit
JSON	JavaScript Object Notation	**WAL**	Write-Ahead Logging
JSONL	JSON Lines	**WCAG**	Web Content Accessibility Guidelines
JWT	JSON Web Token	**XML**	Extensible Markup Language
LLM	Large Language Model		

CHAPTER 1
INTRODUCTION

1.1 Background

Generative AI has moved from a curiosity to a daily work tool in a very short time. Support agents ask a model to draft replies, engineers paste stack traces and configuration files to find a bug, HR teams turn rough notes into offer letters, and analysts ask for help with slow SQL queries. Each of these requests is a **prompt**: a block of text that leaves the company and travels to an AI provider such as AWS Bedrock, Azure AI Foundry, GCP Vertex AI or a locally hosted model.

The problem is that people paste what is in front of them. A configuration file often contains a database password. A support ticket contains a customer's email address, phone number and card number. An incident summary contains internal hostnames and IP addresses. Once that text is sent, the company no longer controls where it is stored, who can read it, or whether it ends up in logs or training data. OWASP now lists **sensitive information disclosure** among the top risks for applications built on large language models [1], and real companies have restricted AI tools after staff pasted confidential source code into a public chatbot [2].

CipherMesh is the answer this project builds to that problem. It is a privacy control plane: a checkpoint that sits between an organisation's applications and every AI model it uses. Every prompt is inspected before it leaves. Secrets are stopped, personal details are hidden, internal context is held for a human to approve, and only safe text is forwarded, to the model that offers the best balance of privacy, quality, cost and speed.

> **IN PLAIN WORDS**
> Think of CipherMesh as airport security for AI prompts. Every bag (prompt) goes through the scanner. Harmless bags go straight through. Bags with a few personal items get those items sealed away. Suspicious bags wait for an officer. Bags with something dangerous never board the plane.

1.2 Problem Statement

Organisations need a way to use many AI providers without leaking secrets or personal data, and without asking every developer to remember every rule. Existing guardrails are usually tied to one cloud vendor, focus on harmful content rather than data leakage, or detect patterns without validating them, which causes false alarms. There is no single, provider-neutral layer that (a) reliably finds credentials and Indian and international personal identifiers, (b) makes a clear, explainable decision, (c) keeps tamper-evident evidence without storing the raw prompt, (d) lets a person review borderline cases, and (e) routes the remaining safe traffic to the most suitable model while respecting data-residency rules such as India's DPDP Act [3].

1.3 Motivation

The project was carried out for the course **21CSE525T Agent Technology**. The course teaches what makes software an *agent*: it perceives its environment, reasons about what it sees, and acts towards a goal [4], [5]. AI privacy is a natural fit for that idea. The environment (prompts, providers, policies, reviewers) is partly observable and keeps changing; the agent must act on every request in milliseconds; several agents must coordinate; and the agent must make self-interested choices about cost and quality. Each course assignment added one of these abilities, and together they became one working product.

1.4 Objectives

1. Design a privacy agent for multi-cloud AI use, with a clear domain, persona and PEAS profile (Assignment 1.1).
2. Build a rule-based reasoner that perceives prompts and triggers condition-action rules (Assignment 1.2).
3. Run the agent concurrently with worker threads, a monitoring daemon and thread-safe state (Assignment 2.1).
4. Let agents exchange typed messages over sockets and react to remote commands (Assignment 2.2).
5. Coordinate several agents through negotiation and a consensus handshake (Assignment 3.1).
6. Choose the best option with a utility function under hard constraints (Assignment 3.2).
7. Unify everything into one tested, secure and accessible product: CipherMesh v2.3, with an API, an agent graph, a web console and a product site.

1.5 Scope

Table 1.1 separates what the project covers from what it deliberately leaves out.

Table 1.1: Scope of the project

In scope	Out of scope
Text prompts sent to AI models through the CipherMesh API or web console	Images, audio and file uploads
25 detection rules for credentials, personal identifiers and internal links	Machine-learning classifiers for free-form confidential text
Four decisions (ALLOW, REDACT, REVIEW, BLOCK) with a human review queue	Single sign-on and per-person reviewer identity
Simulated AWS, Azure and GCP providers, plus real local Ollama on request	Paid production contracts with cloud AI providers
Response-side inspection of model replies	Full data-loss prevention for email, chat and file shares

1.6 Contributions

- **A working product, not a demo script.** A FastAPI gateway, a LangGraph agent, an encrypted evidence vault, a reviewer queue and a Next.js console, deployed on Vercel.
- **Validated detection.** Card numbers are checked with the Luhn algorithm [6] and Aadhaar numbers with the Verhoeff checksum [7], so random digit runs are not mistaken for identifiers.
- **Privacy-aware routing.** A utility function picks the provider, but only among those that satisfy hard rules; prompts containing Aadhaar, PAN or Indian phone numbers go only to India-resident or local models.
- **Every assignment reused.** Each course assignment became a live module of the product (Figure 1.1), and an internal audit fixed twelve gaps that appeared when they were joined.
- **Evidence.** 167 automated tests, a labelled evaluation corpus, concurrency benchmarks and an accessibility audit.

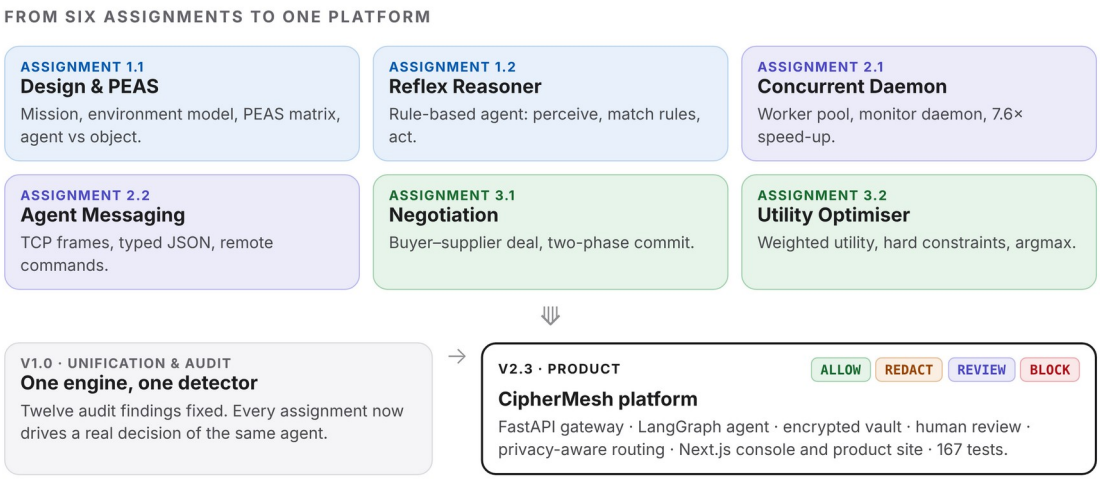


Figure 1.1: How the six assignments grew into one platform

1.7 Organisation of the Report

Chapter 2 reviews related work. Chapters 3 and 4 explain the architecture and every module. Chapters 5 to 10 take each assignment in turn: what was asked, what was delivered, how it works and how well the submission met the rubric. Chapter 11 presents the product and its interface, Chapter 12 the implementation and deployment, and Chapter 13 the testing and results. Chapter 14 concludes. References follow in IEEE style, and the appendices give the API, configuration and a short user guide.

CHAPTER 2
LITERATURE SURVEY

2.1 Data Leakage Through AI Prompts

Large language models can memorise and later repeat parts of the text they see. Carlini *et al.* showed that verbatim training data, including names, phone numbers and addresses, can be extracted from a production language model [8]. Greshake *et al.* showed that text placed inside documents or web pages can steer an AI application into revealing or misusing data, a risk called indirect prompt injection [9]. The OWASP Top 10 for LLM applications groups these concerns under sensitive information disclosure and recommends sanitising data *before* it reaches the model [1]. The common lesson is simple: the safest secret is the one that never leaves.

2.2 Existing Guardrails

Cloud vendors offer their own guardrails. Amazon Bedrock Guardrails can filter topics and mask some personal information inside Bedrock [10]; Azure AI Content Safety focuses on harmful content and prompt attacks [11]. Microsoft Presidio is an open-source toolkit for finding and anonymising personal data [12], and secret scanners such as GitHub secret scanning look for leaked keys in code repositories [13]. Cloud security guides also stress that access keys, service-account keys and shared-access signatures must never be exposed [14], [15], [16].

Each of these solves part of the problem. Vendor guardrails work only inside that vendor's cloud. Presidio finds personal data but does not decide where the text should go. Secret scanners look at code at rest, not at prompts in flight. None of them negotiates a price with several providers or enforces data residency while choosing a model.

2.3 Intelligent Agent Foundations

Russell and Norvig define an agent by its **PEAS** description: performance measure, environment, actuators and sensors [4]. They also classify environments as observable or partially observable, deterministic or stochastic, episodic or sequential, static or dynamic, discrete or continuous, and single- or multi-agent. The simplest agent design is the **simple reflex agent**, which maps the current percept straight to an action through condition-action rules. Wooldridge and Jennings add that agents are autonomous, reactive, proactive and social [5], which separates them from ordinary objects that only run when called.

2.4 Multi-Agent Coordination and Negotiation

The **Contract Net Protocol** lets a manager announce a task, collect bids from contractors and award the work [17]. Faratin, Sierra and Jennings described negotiation decision functions in which each agent concedes over time according to a tactic, until offers cross [18]. Once agents agree, they need to commit reliably; the classic answer is the **two-phase commit** protocol from database systems, in which a coordinator first asks every participant to prepare and only then tells them to commit [19]. Wooldridge's text brings these ideas together for multi-agent systems [20].

2.5 Utility and Decision Making

Multi-attribute utility theory turns several goals (quality, cost, speed) into one comparable number by normalising each attribute and combining them with weights [21]. A key practical rule is to treat some requirements as **hard constraints** that remove options entirely, and to use

the utility score only to rank what remains. CipherMesh uses this rule both in Assignment 3.2 and in its provider router.

2.6 Cryptographic Building Blocks

CipherMesh stores evidence with **Fernet**, an authenticated encryption scheme built on AES-128-CBC and HMAC-SHA256 [22]. The key is derived with **PBKDF2-HMAC-SHA256** [23] at 600,000 iterations, the level recommended by OWASP for this function [24]. Message integrity between agents uses **HMAC** [25]. Card and Aadhaar numbers are validated with the Luhn [6] and Verhoeff [7] check-digit algorithms.

2.7 Regulation

India's Digital Personal Data Protection Act, 2023 sets duties for anyone processing digital personal data of people in India [3]. The EU GDPR restricts transfers of personal data outside approved regions [26], and the NIST AI Risk Management Framework asks organisations to map and manage privacy risks of AI systems [27]. These motivate two CipherMesh features: hiding personal data before it leaves, and routing prompts that contain Indian identifiers (Aadhaar, PAN, Indian phone numbers) only to India-resident or local models.

2.8 Research Gap

Table 2.1 compares CipherMesh with the approaches reviewed above.

Table 2.1: Comparison of CipherMesh with existing approaches

Capability	Vendor guardrails	Presidio	Secret scanners	CipherMesh
Works across many AI providers	No	Yes (library)	Not for prompts	Yes
Blocks credentials in prompts	Partly	No	In code only	Yes, 16 rules
Checksum-validated IDs (card, Aadhaar)	No	Partly	No	Yes
Human review queue	No	No	No	Yes
Residency-aware model routing	No	No	No	Yes
Encrypted, metadata-only evidence	Vendor logs	No	No	Yes
Zero-cost, open stack	No	Yes	Varies	Yes

CHAPTER 3
SYSTEM ARCHITECTURE

3.1 The Four Layers

CipherMesh is organised in four layers that are read from left to right (Figure 3.1). Requests only ever flow in one direction, and a red **trust boundary** separates everything the organisation controls from the outside AI providers.

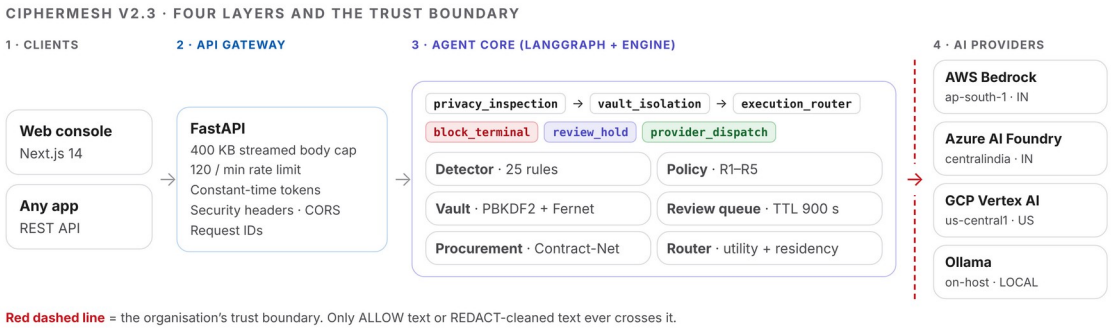


Figure 3.1: CipherMesh system architecture and trust boundary

Table 3.1 summarises what each layer does.

Table 3.1: The four layers

Layer	What it does
1. Clients	The CipherMesh web console, or any application that calls the REST API.
2. API gateway	FastAPI service that checks every request: body-size cap, rate limit, tokens, security headers and request IDs.
3. Agent core	The LangGraph agent and the CipherMesh engine: detection, policy, vault, review, negotiation and routing.
4. AI providers	AWS Bedrock (Mumbai), Azure AI Foundry (Central India), GCP Vertex AI (US) and a local Ollama model. Only ALLOW or REDACT text crosses to them.

3.2 The Agent Core

The decision engine is a **state machine built with LangGraph** [28]. Every prompt walks the same path (Figure 3.2): `privacy_inspection` finds sensitive data and picks a decision, `vault_isolation` seals the evidence, and `execution_router` sends the request to exactly one of three exits. A blocked request ends at `block_terminal`, a held request at `review_hold`, and allowed or cleaned text goes to `provider_dispatch`, which negotiates, routes, calls the model and inspects the reply.

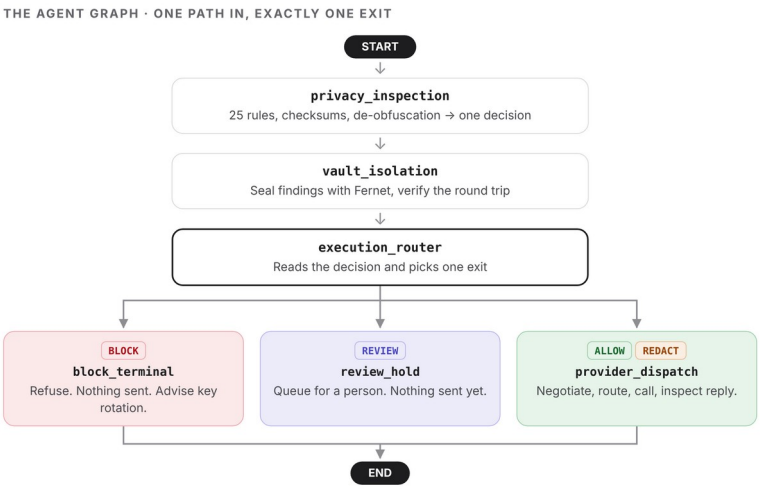


Figure 3.2: The LangGraph state machine: one path in, exactly one exit

The agent's memory is a typed `AgentState` (Table 3.2). Two fields, `logs` and `graph_path`, use append-only reducers, so each node can add to them but never rewrite history. The `raw_prompt` field is cleared as soon as it is no longer needed, so the original text does not linger in memory or in any response.

Table 3.2: Key fields of the typed AgentState

Field	Written by	What it holds
`raw_prompt`	Request	The original text; cleared once no longer needed
`action_decision`, `reasons`, `findings`	privacy_inspection	The decision, the rules that fired and what was found
`processed_prompt`, `safe_preview`	privacy_inspection	The text that may leave, with every finding masked
`vault_token`	vault_isolation	Fernet token of the sealed evidence
`review_id`	review_hold	Ticket for a held prompt
`route`, `routed_provider`, `response`	provider_dispatch	Chosen provider, its negotiated price and the checked reply
`logs`, `graph_path`	Every node (append-only)	The audit trail of the run

3.3 Life of One Request

Figure 3.3 follows one prompt that contains an email address and an Aadhaar number. The detector finds both, so the decision is REDACT. Because Aadhaar is Indian personal data, the router only considers India-resident or local providers and picks AWS Bedrock in Mumbai at a negotiated price. Only the masked text crosses the trust boundary, and the reply is inspected on the way back.

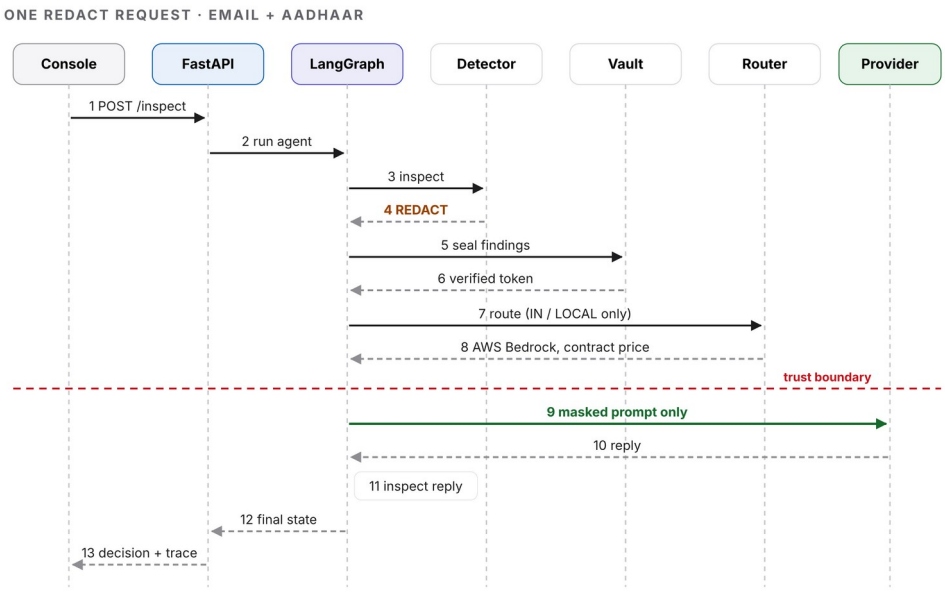


Figure 3.3: Sequence of a single REDACT request, from console to provider and back

3.4 Technology Stack

Table 3.3 lists the main technologies and why each was chosen.

Table 3.3: Technology stack

Area	Technology	Why it was chosen
Agent orchestration	LangGraph 1.2 StateGraph	Explicit nodes and conditional edges make the agent auditable
API	FastAPI 0.162, Pydantic 2, Uvicorn	Typed validation, automatic OpenAPI docs, async I/O [29], [30]
Encryption	cryptography: PBKDF2 + Fernet, MultiFernet	Standard, audited primitives; built-in key rotation
Storage	SQLite (WAL when file-backed; in-memory on the hosted API)	Zero-cost, safe concurrent reads and writes [31]
Frontend	Next.js 14, React 18, TypeScript, Tailwind CSS	Server rendering, strict types, fast styling [32]
Quality	pytest, ESLint, tsc in CI; Playwright and axe-core in review passes	Automated checks on every push to main [33]
Delivery	Docker, GitHub Actions, Vercel	One-command local run and free hosting

3.5 Design Principles

- **Fail closed.** If anything is unclear (a full queue, an expired review, a bad frame), nothing is sent.
- **Keep the least data.** The raw prompt is never logged; the agent graph fingerprints prompts with a keyed HMAC, and audit events keep only short hashes and metadata.
- **Explain every decision.** Each result names the rule that fired, the graph path and the provider.
- **One engine.** Every feature calls the same detector and policy, so a rule fixed once is fixed everywhere.

CHAPTER 4
MODULE DESCRIPTIONS

This chapter explains each module in plain terms: what it is for, how it works, and where it lives in the code. Figure 4.1 is the map.

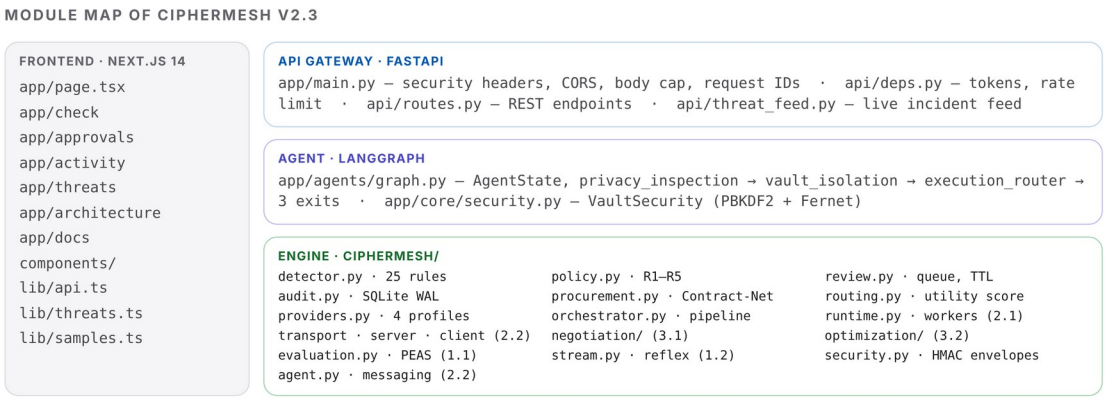


Figure 4.1: Module map of CipherMesh v2.3

4.1 API Gateway

The gateway is the front door (`backend/app/main.py`, `api/deps.py`, `api/routes.py`). Before the agent sees anything, it enforces a **400 KB body limit while the body is still streaming**, so a chunked upload cannot slip past the check. A sliding-window **rate limit** allows 120 requests per minute per client. Tokens are compared in constant time to avoid timing leaks. Every response carries a request ID and security headers (a Content-Security-Policy with `frame-ancestors 'none'`, `nosniff`, `X-Frame-Options: DENY`, `no-referrer` and `no-store`; HSTS is added by the Vercel edge), and CORS never allows credentials. Validation errors never echo the prompt back.

4.2 Detector

The detector (`ciphermesh/detector.py`) runs **25 rules** grouped into twelve families, D1 to D12 (Table 4.1). Each rule has a confidence and a severity. Before matching, the text is also **de-obfuscated**: spaced-out characters are joined, base64 blocks are decoded, and tricks such as `name [at] company [dot] com` are normalised, so simple disguises do not work. Two families are validated with checksums, which removes most false alarms on random numbers.

Table 4.1: Detection families

Family	What it finds	Severity	Example (synthetic)
D1	AWS access keys and secret keys	Critical	`AKIA` followed by 16 characters
D2	Private keys, GCP service-account keys	Critical	`BEGIN PRIVATE KEY` header
D3	Azure SAS signatures, storage account keys	Critical	`?sig=...` in a blob URL
D4	GitHub, OpenAI-style, Slack, Google, Stripe tokens, JWTs, DB URLs, passwords	Critical	`postgres://user:pass@host`
D5	Email addresses	Sensitive	`name@example.com`
D6	Indian and international phone numbers	Sensitive	`+91 98765 43210`
D7	IPv4 addresses (version strings ignored)	Sensitive	`10.20.30.40`

Family	What it finds	Severity	Example (synthetic)
D8	Internal or corporate URLs	Confidential	`https://payments.internal/`
D9	Payment cards, Luhn-validated	Sensitive	16-digit card that passes Luhn
D10	Aadhaar numbers, Verhoeff-validated	Sensitive	12 digits that pass Verhoeff
D11	PAN (Indian tax ID)	Sensitive	`ABCPE1234F`
D12	US Social Security numbers	Sensitive	`078-05-1120`

4.3 Policy Engine

The policy engine (`ciphermesh/policy.py`) turns findings into one decision using five rules, R1 to R5. When several rules match, **the strictest wins** (Figure 4.2). An emergency lockdown, which can be triggered remotely, overrides everything and blocks all traffic.

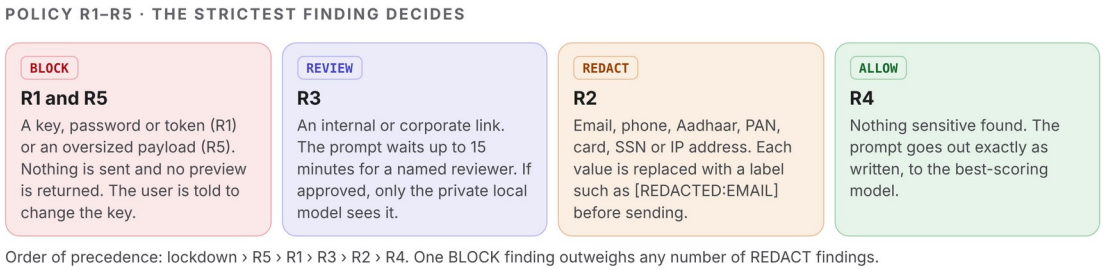


Figure 4.2: Policy R1–R5 and the order of precedence

4.4 Evidence Vault

`VaultSecurity` (`app/core/security.py`) seals the findings and the execution details of every request. A master seed and a per-deployment salt are stretched with PBKDF2-HMAC-SHA256 at 600,000 iterations into a Fernet key [23], [24]. The vault encrypts the record, immediately decrypts it again to prove the round trip works, and returns a token. The audit log holds only metadata (decision, finding categories, reasons, timings) and the sealed ciphertext, never the prompt itself. **MultiFernet** keeps older seeds readable during key rotation, so evidence sealed last month can still be audited after the key changes. In production the service refuses to start without a strong seed and salt.

4.5 Review Queue

A prompt with an internal link is not blocked outright, because it may be harmless; it waits for a person. The review queue (`ciphermesh/review.py`) holds the text **in memory only**, for a time-to-live of 900 seconds, with at most 1,000 items. Each item can be decided once, and the reviewer's name is recorded. On approval the text is **inspected again** (a lockdown may have started meanwhile) and is released only to the private local model, never to an outside cloud. If the queue is full, new items are blocked rather than dropped silently.

4.6 Procurement and Routing

Before an allowed prompt is sent, the agent buys capacity. The procurement module (`procurement.py`) negotiates a contract with **each provider** using the Contract-Net economics of Assignment 3.1 [17], scaled to that provider's list price, and caches the committed price for an hour. For example, AWS Bedrock's $2.80 list price settles at $2.28 per thousand requests. The router (`routing.py`) then scores every provider with a weighted utility, the in-domain version of Assignment 3.2:

$$U = 0.30\,Q + 0.25\,C + 0.20\,L + 0.15\,A + 0.10\,P$$

Here Q is quality, C is cost (cheaper scores higher), L is latency, A is availability and P is privacy fit (100 for local, 80 for an in-country cloud, 50 otherwise); each term runs from 0 to 100. Hard constraints are applied first: maximum price, maximum latency, minimum quality and availability, and **data residency**. If the prompt contained Aadhaar, PAN or an Indian phone number, only India-resident or local providers are allowed. A circuit breaker skips a provider that keeps failing and falls back to the next best one. Table 4.2 lists the four provider profiles.

Table 4.2: Provider profiles used by the router

Provider	Region	Residency	List price / 1k	p95 latency	Quality
AWS Bedrock	ap-south-1	IN	$2.80	650 ms	4.6 / 5
Azure AI Foundry	centralindia	IN	$3.30	600 ms	4.7 / 5
GCP Vertex AI	us-central1	US	$2.20	580 ms	4.7 / 5
Ollama (local)	on-host	LOCAL	$0.40	1,200 ms	3.4 / 5

Prices and latencies are illustrative values for the simulated providers, not published rates.

4.7 Supporting Modules

Table 4.3 shows where the code from each assignment lives in the product.

Table 4.3: Modules carried forward from the assignments

Module	Origin	Role in the product
`evaluation.py`	Assignment 1.1	Measures the PEAS performance goals on a labelled corpus
`stream.py`	Assignment 1.2	Stream reasoner: reads JSONL requests and logs perceive-reason-act steps
`runtime.py`	Assignment 2.1	Bounded worker pool and monitor daemon around the orchestrator
`agent.py`, `transport.py`, `server.py`, `client.py`, `security.py`	Assignment 2.2	Message-handling agent and signed, length-prefixed messaging: state sync, remote lockdown, pattern updates
`negotiation/`, `procurement.py`	Assignment 3.1	Buyer–supplier negotiation and two-phase commit; prices every provider
`optimization/`, `routing.py`	Assignment 3.2	Constrained utility maximisation; chooses the provider

4.8 Threat Intelligence Feed

The product site includes an AI threat dashboard. Its curated dataset (`frontend/lib/threats.ts`) holds 38 documented AI security incidents from 2023 to 2026, each with a source link. A live panel calls `/api/v1/threats/live`, which reads the AI Incident Database RSS feed [34], keeps only security-related headlines, caches the result for 30 minutes and refuses XML documents that declare a DTD, which protects against XML entity attacks.

4.9 Web Console and Product Site

The frontend is a Next.js 14 application with two shells: a **product site** (overview, threat intelligence, how it works, documentation) and a **console** (check, approvals, activity). Chapter 11 presents both in detail.

CHAPTER 5
ASSIGNMENT 1.1: PROJECT IDEATION, ENVIRONMENT MODELING AND PEAS MATRIX

THE ASSIGNMENT IN ONE LINE
Pick a real problem, describe the agent that would solve it in the PEAS language, classify its environment, and explain why it is an agent and not just an object.

5.1 What Was Asked

The first assignment set the foundation for the whole course. Each student had to choose a domain where an intelligent agent is genuinely useful, describe the people it serves, and specify the agent formally before writing any code. The marking scheme had three parts (Table 5.1).

Table 5.1: Assignment 1.1 rubric

Criterion	Marks	What the examiner looks for
Domain and persona clarity	1	A real problem, a clear user, and why an agent fits
PEAS profile completeness	2	Every element of performance, environment, actuators and sensors, specific to the domain
Agent versus object contrast	2	A reasoned comparison showing what makes the design an agent
Total	**5**	

5.2 What Was Delivered

The submission was a design specification, *CipherMesh: Multi-Cloud LLM Privacy Agent*. It covered the mission, stakeholders, a zero-cost technology stack, the PEAS matrix, seven environment classifications, the perceive-reason-act loop, the first version of policy rules R1 to R5, an eight-point agent-versus-object table, a production roadmap and research sources from OWASP, the three cloud vendors, Presidio and Ollama [1], [14], [15], [16], [12], [35].

5.3 Domain and Personas

The domain is **privacy for enterprise AI use**. The agent persona is an autonomous privacy gateway between applications and model providers that, for every request and response, perceives the content, detects sensitive entities, judges the risk and picks one of four actions. The specification named three stakeholder groups (Table 5.2), because clear users make the requirements concrete.

Table 5.2: Stakeholders defined in the specification

Stakeholder	What they need from the agent
Developers	Avoid accidentally sending credentials, internal URLs, customer records or code secrets to an external model
Customers	Real-time protection when entering personal data into AI chatbots and support assistants
Security and privacy teams	Consistent, provider-neutral policy enforcement and auditable decisions

The specification also drew a clear boundary: CipherMesh inspects application-layer text before it is sent and after the model replies, and it does not claim to replace identity management, secret managers, encryption or provider-native guardrails.

5.4 The PEAS Profile

PEAS describes an agent by what success means, where it lives, what it can do and what it can perceive [4]. Figure 5.1 shows the profile as it stands in the product today. Each performance measure is a number that can be checked, which is why they later became the metrics computed by `evaluation.py`.

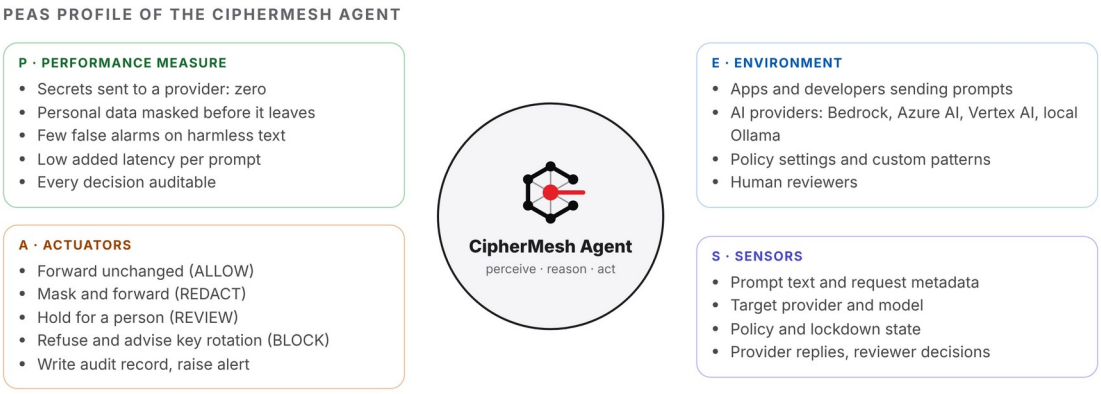


Figure 5.1: PEAS profile of the CipherMesh agent

5.5 Classifying the Environment

Table 5.3 gives the seven environment properties from the specification.

Table 5.3: Environment classification (values from the specification, explained in plain words)

Property	Value	Why, in plain words
Observability	Partially observable	The agent sees the prompt, not the intent behind it or what the provider does later
Determinism	Stochastic	Provider latency, failures and model replies vary from call to call
Episodes	Sequential	Earlier events matter: a lockdown or a learned pattern changes later decisions
Dynamics	Dynamic	New prompts, reviews and commands arrive while the agent is deciding
State space	Mixed	Discrete decisions, but continuous prices, latencies and scores
Agents	Multi-agent	Providers, reviewers and peer agents all act in the same environment
Knowledge	Partly known	Credential formats are known; new token formats appear over time

5.6 Agent Versus Object

An object waits to be called and does exactly what the caller asks. An agent pursues a goal on its own [5]. The specification made the contrast concrete with eight aspects (Table 5.4).

Table 5.4: Agent versus object, as argued in the submission

Aspect	CipherMesh autonomous agent	Standard object-oriented program
Control	Owns a continuous perceive-decide-act loop	Waits for an external caller to invoke methods
Situated behaviour	Operates at a live trust boundary between application and models	Represents data and functions with no place in an environment
Decision authority	Chooses ALLOW, REDACT, REVIEW or BLOCK from policy and percepts	Executes the operation chosen by the surrounding code
Initiative	Stops unsafe transmission and warns the user without a separate command	Does not act unless orchestrated externally
State	Percepts, findings, policy version, obligations, approvals, audit context	Data fields and lifecycle state
Communication	Exchanges typed messages with apps, providers, humans and other agents	Calls known interfaces or returns values
Adaptation	Later updates policies and plans from feedback and new detector knowledge	Changes only through explicitly coded logic
Example	Detects a cloud key inside a prompt and blocks it autonomously	A scanner returns matches; another component must decide

5.7 Submission Analysis

Table 5.5 maps the evidence in the submission to each rubric criterion.

Table 5.5: Rubric coverage for Assignment 1.1

Criterion	Evidence in the submission	Assessment
Domain and persona	Mission, persona, three stakeholder groups, OWASP and vendor guidance	Fully met; the problem is real and current
PEAS completeness	All four elements, each specific and measurable	Fully met; later turned into code
Agent vs object	Eight-aspect table plus the perceive-reason-act loop	Fully met; goes beyond the minimum

- **Strengths.** The design was specific enough to build from: the rules R1 to R5, the four decisions and the provider list all survived into v2.3.
- **What changed later.** The size rule (R5) and lockdown were refined, the twelve-family detector replaced the first eight categories, and the performance measures became an automated evaluation (Chapter 13).

CHAPTER 6
ASSIGNMENT 1.2: RULE-BASED ENVIRONMENTAL REASONER

THE ASSIGNMENT IN ONE LINE
Turn the design into running code: an agent that reads requests, recognises what is in them and fires the right rule.

6.1 What Was Asked

Students had to implement a working reasoner that perceives its environment through simulated sensor input, parses that input safely, and triggers condition-action rules. The rubric (Table 6.1) rewarded code that runs cleanly, robust parsing, and correct rule firing.

Table 6.1: Assignment 1.2 rubric

Criterion	Marks	What the examiner looks for
Code execution and syntax	2	Runs end to end without errors; clean, readable code
Sensory parsing	1.5	Input is read and validated; bad input is handled, not crashed on
Condition-action triggering	1.5	The right rule fires for each situation, with a clear action
Total	**5**	

6.2 What Was Delivered

Table 6.2 lists the files submitted.

Table 6.2: Assignment 1.2 deliverables

File	Purpose
`ciphermesh_agent.py`	The agent; Python standard library only, so it runs anywhere with no installation
`mock_llm_requests.jsonl`	The simulated sensor stream: seven requests, one deliberately malformed
`execution_log.txt`	The agent's full perceive-reason-act trace for the stream
`README.md`	How to run it and how each rule maps to the design

6.3 How It Works

The program is a **simple reflex agent** [4]. It keeps no memory between requests: each line is perceived, matched and acted on independently, so the same input always gives the same output (Figure 6.1).

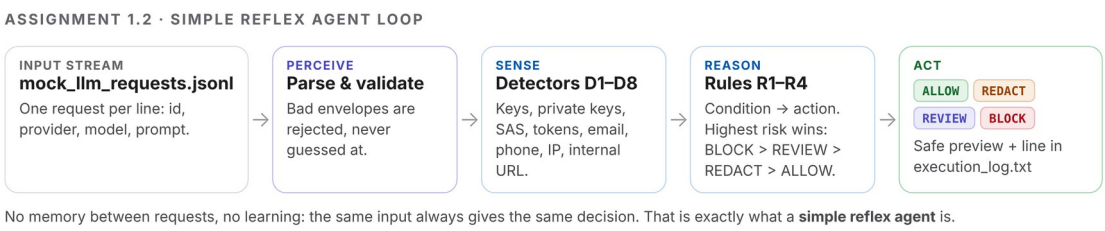


Figure 6.1: The simple reflex loop implemented in Assignment 1.2

1. **Perceive.** Each JSON line is parsed and checked: it needs an ID, a known provider, a model and a prompt. A line that fails is rejected with a reason and the stream carries on.
2. **Sense.** Eight detectors (D1 to D8) look for AWS keys, private keys, Azure SAS signatures, API tokens, emails, Indian phone numbers, IP addresses and internal URLs.
3. **Reason.** Rules map findings to actions: a critical secret means BLOCK (R1), personal data means REDACT (R2), an internal URL means REVIEW (R3) and nothing means ALLOW (R4). If several apply, the highest risk wins.
4. **Act.** The agent prints the decision, a *safe preview* with every finding masked, and the instruction for the developer, such as "revoke and rotate the exposed credential".

6.4 Execution Walkthrough

Table 6.3 is taken directly from `execution_log.txt`. Note request 6: an IP address alone would only be redacted, but together with an internal URL the stricter REVIEW wins. Note also input 7, which names a provider that does not exist and is rejected instead of guessed at.

Table 6.3: Results of the seven-line input stream

Input	Provider	Findings	Decision
REQ-001	ollama-local	None	**ALLOW**
REQ-002	azure-ai	D5 email, D6 Indian phone	**REDACT**
REQ-003	aws-bedrock	D1 AWS access key ID	**BLOCK**
REQ-004	gcp-vertex	D2 private key header	**BLOCK**
REQ-005	azure-ai	D3 Azure SAS signature	**BLOCK**
REQ-006	gcp-vertex	D7 IP address, D8 internal URL	**REVIEW**
INPUT-07	unknown-cloud	Envelope invalid	**REJECTED**

The log entry for request 3 shows all three steps and the masked preview:

```
[03] PERCEIVE | id=REQ-003 provider=aws-bedrock prompt_sha256=dc25e8af7b26
     REASON | D1 matched | type=AWS_ACCESS_KEY_ID severity=CRITICAL
     ACT    | BLOCK | Do not transmit; revoke/rotate exposed credential
     SAFE_PREVIEW | aws_access_key_id=[AWS_ACCESS_KEY_REDACTED]
```

6.5 How It Lives On in v2.3

The reflex loop survives as `ciphermesh/stream.py`, now running on the shared detector, and its rule structure is the core of today's `policy.py`. The eight detectors grew into 25 rules in twelve families. Two details were corrected during the v1.0 audit: the GitHub token rule used `ghp-` where real tokens use `ghp_`, and some token formats blocked here were not yet ported to the packaged gateway. Both were fixed and are covered by tests.

6.6 Submission Analysis

Table 6.4 maps the evidence in the submission to each rubric criterion.

Table 6.4: Rubric coverage for Assignment 1.2

Criterion	Evidence	Assessment
Execution and syntax	Runs with the standard library only; full log produced	Fully met
Sensory parsing	Envelope validation; malformed input 7 rejected safely	Fully met
Condition-action triggering	All four actions demonstrated; precedence shown in REQ-006	Fully met

- **Strengths.** Safe previews mean the log itself never leaks a secret, and only a short SHA-256 fingerprint of each prompt is recorded.
- **Limits.** Regular expressions alone cannot catch a secret broken into pieces or an identifier spelled out in words; this is measured honestly in Chapter 13.

CHAPTER 7
ASSIGNMENT 2.1: CONCURRENT TASK DAEMONIZATION

THE ASSIGNMENT IN ONE LINE
Make the agent handle many requests at once, watch its own health in the background, and never corrupt shared data.

7.1 What Was Asked

A real gateway receives many prompts at the same time. This assignment asked for a multi-threaded worker design, a background **daemon** that monitors the system, and proof that shared state stays correct under concurrency (Table 7.1).

Table 7.1: Assignment 2.1 rubric

Criterion	Marks	What the examiner looks for
Multi-threading / worker infrastructure	2	A real pool of workers fed by a queue, with measurable benefit
Daemon monitoring	1.5	A background thread that observes the system and stops cleanly
Thread safety	1.5	Locks and safe hand-offs; no lost or duplicated work
Total	**5**	

7.2 What Was Delivered

Table 7.2 lists the files submitted.

Table 7.2: Assignment 2.1 deliverables

File	Purpose
`ciphermesh_concurrent_agent.py`	Worker pool, monitor daemon, audit writer and shutdown logic
`benchmark_concurrency.py`	Runs 80 requests with 25 ms simulated provider wait at 1, 2, 4 and 8 workers
`tests/test_concurrency.py`	Checks no task is lost or processed twice and shutdown is clean
`logs/` (7 files)	Console log, latency log, monitor metrics, audit events and benchmark results (CSV, JSON, text)
Report (DOCX/PDF) with 7 figures	Architecture, sequence, thread safety, benchmarks and decision mix

7.3 How It Works

Figure 7.1 shows the design. The main loop is a **producer**: it parses requests and puts them in a bounded `queue.Queue` [36]. Because the queue has a maximum size, a flood of requests makes the producer wait instead of exhausting memory; this is called back-pressure. A pool of **worker threads** takes items off the queue and runs the same detect, decide and protect steps as Assignment 1.2. Results go to one queue and audit events to another, which a single writer thread appends to a privacy-safe JSONL file.

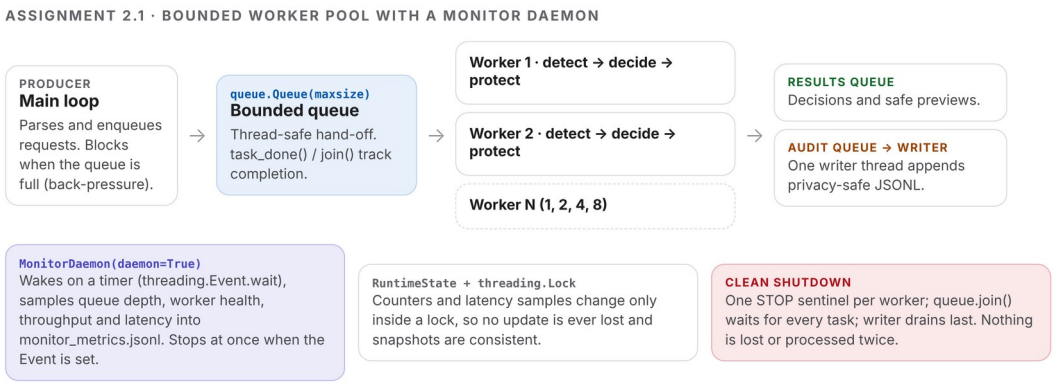


Figure 7.1: Bounded worker pool, monitor daemon and safe shutdown

The **MonitorDaemon** is a thread created with `daemon=True`. It wakes on a timer using `threading.Event.wait`, samples queue depth, worker health, throughput and latency, and writes them to `monitor_metrics.jsonl`. Setting the event stops it immediately, so shutdown never waits for a sleeping monitor. Table 7.3 lists the thread-safety mechanisms.

Table 7.3: Thread-safety mechanisms

Risk	Mechanism	Why it works
Two threads update a counter at once	`threading.Lock` around every update	Only one thread changes shared state at a time, so no update is lost
Producer and workers share work items	`queue.Queue` (internally locked)	Each item is handed to exactly one worker
Knowing when all work is done	`task_done()` and `join()`	The main thread waits until every item is finished
Stopping workers cleanly	One STOP sentinel per worker	Each worker exits after the real work, never in the middle
Shared rules read by all workers	Immutable tuples and frozen dataclasses	Data that cannot change needs no lock

7.4 Benchmark Results

The benchmark sent 80 requests, each with a simulated 25 ms provider wait. Figure 7.2 and Table 7.4 show the outcome. With one worker the pool is as fast as plain sequential code; doubling the workers roughly doubles the throughput, reaching **7.61× at eight workers** (302 requests per second against 40).

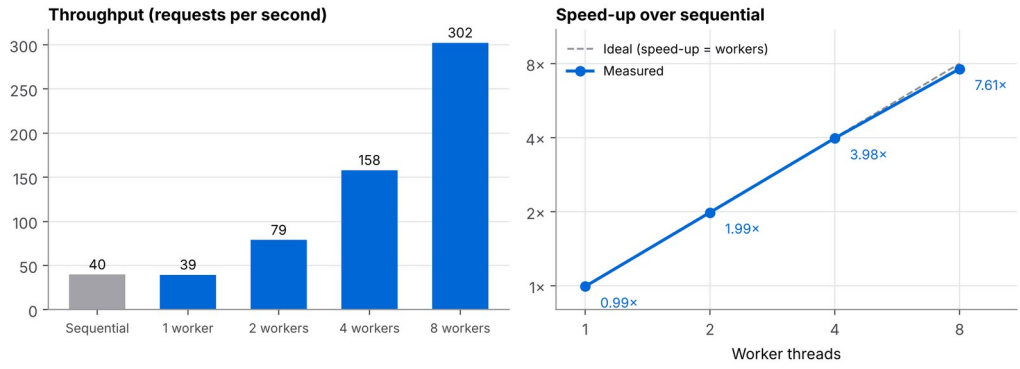


Figure 7.2: Throughput and speed-up as workers are added (80 requests, 25 ms I/O each)

Table 7.4: Benchmark summary

Configuration	Wall time	Throughput	Avg. end-to-end	p95 end-to-end	Speed-up
Sequential	2,015 ms	39.7 req/s	25 ms	25 ms	1.00×
1 worker	2,026 ms	39.5 req/s	1,020 ms	1,915 ms	0.99×
2 workers	1,015 ms	78.9 req/s	518 ms	961 ms	1.99×
4 workers	506 ms	158.0 req/s	265 ms	479 ms	3.98×
8 workers	265 ms	302.3 req/s	139 ms	252 ms	7.61×

WHY THREADS HELP HERE, AND WHEN THEY WOULD NOT

Python's global interpreter lock lets only one thread run Python code at a time [37]. Threads still help because most of each request is *waiting* for the provider, and waiting threads release the lock. The submission says this honestly: the speed-up is for I/O-bound work, not for CPU-heavy regex matching. (The average end-to-end latency is high for one worker because all 80 requests are queued at once and wait their turn.)

7.5 How It Lives On in v2.3

In the first prototype the concurrent agent had its own copy of the detector, so a rule fixed in one place was not fixed in the other. The v1.0 audit moved concurrency into `ciphermesh/runtime.py`, which drives the single shared orchestrator. The audit also found that a handler could call a provider while holding the agent lock, which would have serialised the workers; handlers now run after the lock is released.

7.6 Submission Analysis

Table 7.5 maps the evidence in the submission to each rubric criterion.

Table 7.5: Rubric coverage for Assignment 2.1

Criterion	Evidence	Assessment
Worker infrastructure	Bounded queue, 1–8 workers, measured 7.61× speed-up	Fully met, with data
Daemon monitoring	Event-driven daemon thread writing metrics	Fully met
Thread safety	Locks, sentinels, join; tests prove no loss or duplication	Fully met

- **Strengths.** Every claim is backed by a log or a measurement, and the benchmark states its own limits.
- **Limits.** The prototype had its own copy of the detector; this was merged into the shared engine in v1.0.

CHAPTER 8
ASSIGNMENT 2.2: DISTRIBUTED AGENT MESSAGING

THE ASSIGNMENT IN ONE LINE

Let two CipherMesh agents talk over the network with well-defined messages, and let one agent change the other's behaviour from a distance.

8.1 What Was Asked

Agents are social: they exchange information and requests [5]. This assignment asked for socket or RPC communication between agents, a reliable message format, and *proactive* handling of remote commands, meaning the receiving agent changes what it does because of a message (Table 8.1).

Table 8.1: Assignment 2.2 rubric

Criterion	Marks	What the examiner looks for
Socket stability	2	Connections that handle partial reads, timeouts and bad peers
Serialization and transport	1.5	A clear, versioned message format that is validated
Proactive remote handling	1.5	Commands that change the receiving agent's behaviour
Total	**5**	

8.2 What Was Delivered

The submission (package version 0.2.0) contained a client, Agent-A, and a TCP listener, Agent-B, on `127.0.0.1:9052`, a typed message model, an SQLite audit log and 22 passing tests. Every exchange is one request and one response on a fresh connection, which keeps the protocol easy to reason about.

8.3 The Message Format

TCP delivers a stream of bytes, not separate messages, so the receiver needs to know where one message ends. Each frame therefore starts with a **4-byte big-endian length**, followed by exactly that many bytes of UTF-8 JSON (Figure 8.1). The JSON is a `WireMessage` with a schema version, message ID, type, sender, recipient, timestamp, correlation ID, authentication token and payload, validated by Pydantic before anything is done with it.

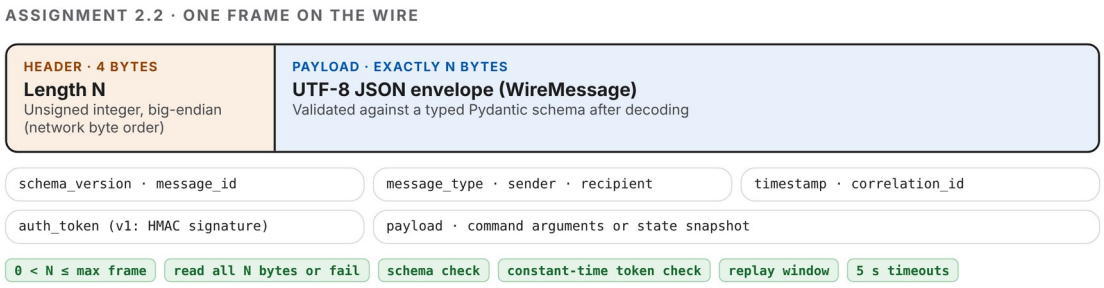


Figure 8.1: Length-prefixed frame and the WireMessage envelope

8.4 One Command, Start to Finish

Figure 8.2 follows a single command from Agent-A to Agent-B and back.

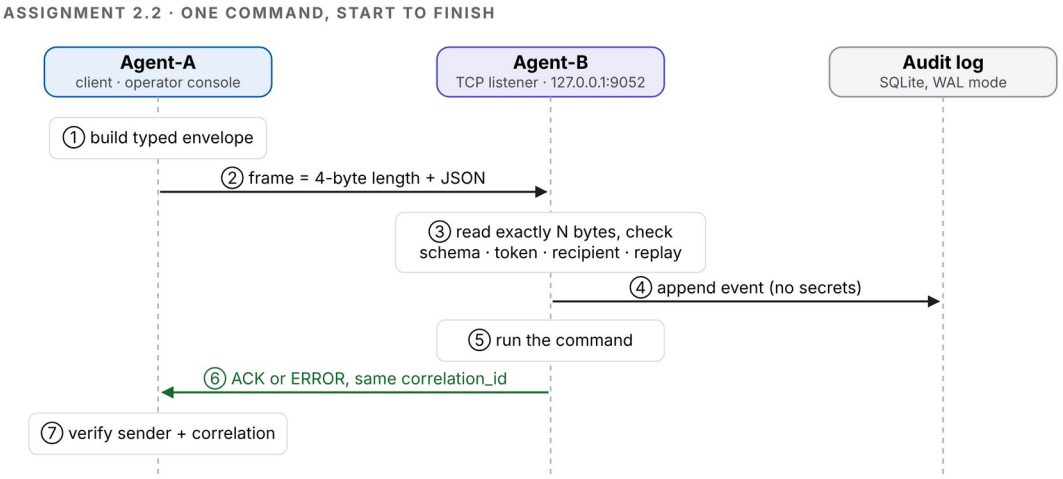


Figure 8.2: Message sequence for one remote command

Agent-B reads exactly N bytes or gives up, then checks the schema, the token (with `hmac.compare_digest`, which takes the same time whether the token is right or wrong), the recipient and a replay-ID window that rejects repeated messages. Only then does it act, record the event without any secret, and reply with the same correlation ID so Agent-A can match the answer to its question.

Table 8.2 lists what Agent-B understands. `STATE_SYNC` is a message type of its own; the other three are commands carried inside a `COMMAND_INSTRUCTION` message.

Table 8.2: Messages and commands, and what they make Agent-B do

Message or command	Effect on the receiving agent
`STATE_SYNC`	Shares allowed state fields with a monotonic revision number; stale revisions are rejected with an ERROR
`TRIGGER_EMERGENCY_LOCKDOWN`	Switches Agent-B into lockdown: every prompt is blocked; there is no remote unlock
`UPDATE_REDACTION_PATTERNS`	Adds custom patterns (at most 32, each at most 128 characters); empty-match patterns refused
`REQUEST_TELEMETRY_DUMP`	Returns decision counts and timings, never prompts or findings

8.5 Safeguards

- **Bounded frames.** The length must be above zero and below a configured maximum, so one message cannot exhaust memory.
- **Timeouts.** Connect, read and write each time out after 5 seconds, so a silent peer cannot hang the agent.
- **Bounded concurrency.** The listener serves at most 32 connections at once.
- **Audit.** Every message is logged to SQLite in write-ahead-logging mode [31], which allows safe concurrent reads.

8.6 How It Lives On in v2.3

The v1.0 audit noted that a shared token only proves the sender knows a string that travels in every frame, and does not protect the message from being changed. The messaging layer now signs a canonical form of every envelope with **HMAC-SHA256** [25], signs replies too,

enforces a 120-second freshness window and keeps a replay cache. Confidentiality on an untrusted network still needs TLS, which is listed as future work.

8.7 Submission Analysis

Table 8.3 maps the evidence in the submission to each rubric criterion.

Table 8.3: Rubric coverage for Assignment 2.2

Criterion	Evidence	Assessment
Socket stability	Exact reads, size limits, 5 s timeouts, 32-connection cap	Fully met
Serialization and transport	Length prefix plus versioned, validated JSON envelope	Fully met
Proactive handling	Lockdown and pattern updates change live behaviour	Fully met; 22 tests

- **Strengths.** The receiver never trusts a frame until size, schema, token, recipient and freshness all pass.
- **Limits.** The shared token proved knowledge but not integrity; v1.0 replaced it with HMAC signatures.

CHAPTER 9
ASSIGNMENT 3.1: MULTI-AGENT COORDINATION

THE ASSIGNMENT IN ONE LINE
Let self-interested agents agree on a price through negotiation, then lock in the deal so that both sides are sure it happened.

9.1 What Was Asked

The third unit moved from one agent to several. The task was to run multiple agent instances that negotiate with each other and then confirm the result through a consensus handshake (Table 9.1).

Table 9.1: Assignment 3.1 rubric

Criterion	Marks	What the examiner looks for
Multi-instance stability	1.5	Several agent instances run together without interfering
Negotiation protocol	2	A defined sequence of offers with a clear rule for agreeing
Consensus handshake	1.5	Both parties confirm the same final agreement
Total	**5**	

9.2 What Was Delivered

The submission (version 0.3.0) added a `negotiation` package with three agents, a protocol, a transport and a demo, plus 31 passing tests. In the scenario, CipherMesh buys privacy-inspection capacity: a **buyer** needs 1,000 inspected requests, an **inspection supplier** sells them, and a **coordinator** runs the final handshake (Table 9.2).

Table 9.2: The three agents

Agent	Wants	Will not go past
`WORKLOAD_BUYER`	A low price; ideal $112 per 1,000	$152.80 (utility 40)
`INSPECTION_SUPPLIER`	A high price; ideal $176 per 1,000	$130.40 (utility 40)
`NEGOTIATION_COORDINATOR`	A correct, confirmed agreement	Never reports a partial deal as success

9.3 The Protocol

The negotiation follows the Contract Net idea [17] and then a two-phase commit [19] (Figure 9.1). The buyer sends a call for proposals (CFP), the supplier proposes, and the two exchange counter-offers until one side accepts. The coordinator then sends PREPARE with a SHA-256 digest of the agreed terms; both agents check the digest and reply READY; only then does the coordinator send COMMIT, and the deal stands when both acknowledge.

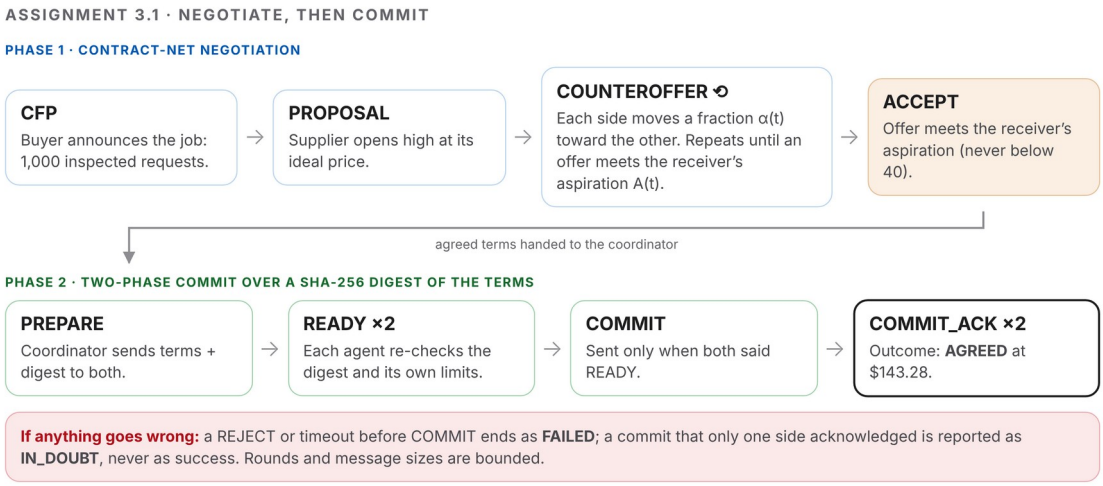


Figure 9.1: Negotiation followed by a two-phase commit

9.4 How Each Agent Decides

Each agent turns a price into a utility between 0 and 100. For the buyer, cheaper is better; for the supplier, dearer is better:

$$U_B(p) = 100 \cdot \mathrm{clip}\left(\frac{180 - p}{180 - 112}\right)$$

$$U_S(p) = 100 \cdot \mathrm{clip}\left(\frac{p - 100}{176 - 100}\right)$$

An agent accepts an offer when its utility is at least its current **aspiration**, which starts high and falls by 9 points per turn but never below the reservation level of 40. Each counter-offer moves a fraction α of the way towards the other side's last offer, and that fraction grows over time, so both sides become more flexible as the talks go on [18]:

$$x_{t+1} = x_t + \alpha(t)\,(y_t - x_t)$$

$$A(t) = \max(40,\ 92 - 9t)$$

$$\alpha_B(t) = \min(0.40,\ 0.12 + 0.035\,t) \qquad \alpha_S(t) = \min(0.42,\ 0.14 + 0.04\,t)$$

Here x is the agent's own last offer, y is the other side's last offer and t is the turn number; clip(·) limits a value to the range 0 to 1, and the subscripts B and S stand for buyer and supplier.

9.5 The Negotiation Trace

Table 9.3 and Figure 9.2 show the five turns, and Figure 9.3 shows both utilities at the agreed price. At turn 4 the buyer offers $140.83, worth 53.7 to the supplier, below its aspiration of 56, so it refuses. At turn 5 the buyer offers $143.28, worth 56.95 to the supplier, above its aspiration of 47, so it accepts. The buyer keeps a utility of 54.00. Both are above 40, and the price sits inside the zone where agreement is possible.

Table 9.3: Offers per turn (price per 1,000 requests)

Turn	Buyer counter-offer	Supplier offer	Outcome
1	$121.92	$166.27	Continue
2	$130.35	$158.37	Continue
3	$136.65	$152.72	Continue
4	$140.83	$149.15	Continue
5	$143.28	Accepts	Agreed at $143.28

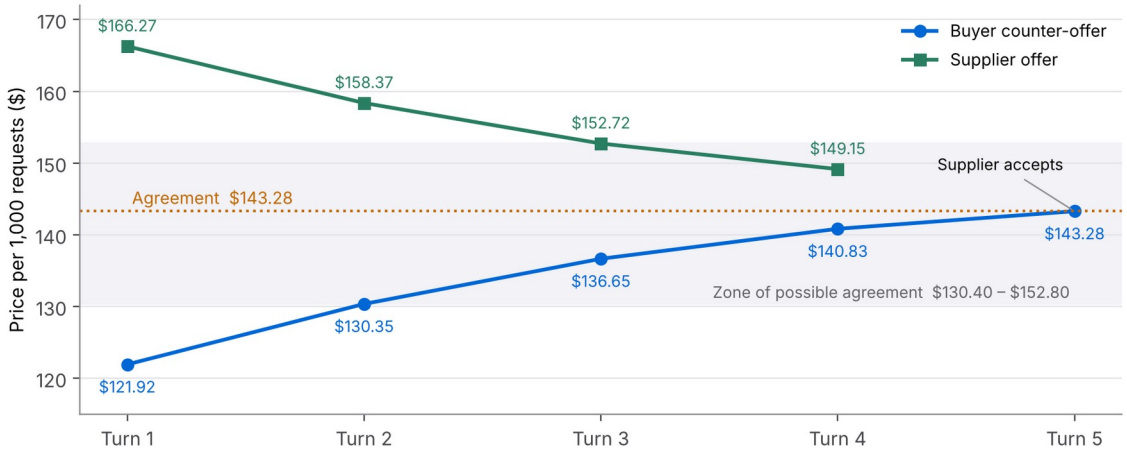


Figure 9.2: Offers converge inside the zone of possible agreement

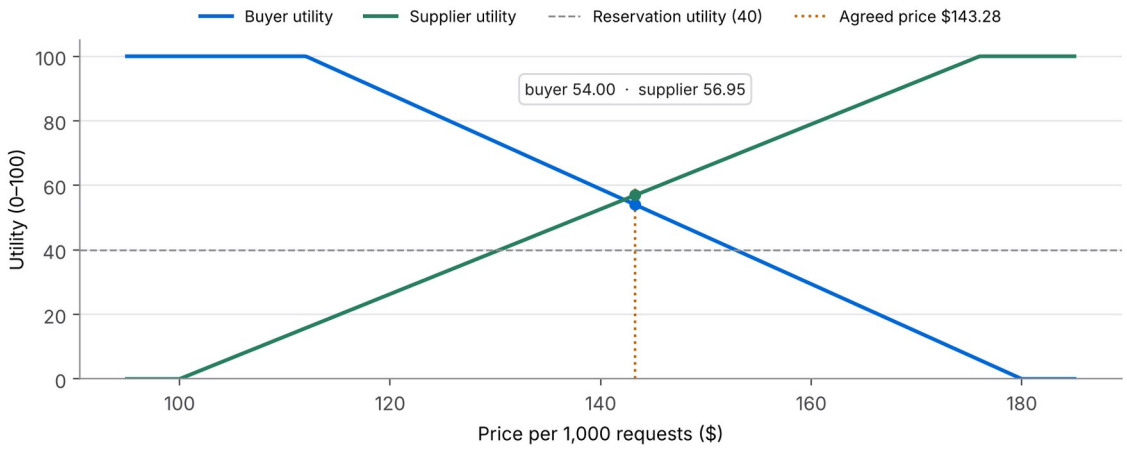


Figure 9.3: Buyer and supplier utility across prices; both stay above the reservation level

The full run used **20 messages and 21 timeline events**. If a reply had been missing or invalid, or one party had acknowledged the commit and the other had not, the coordinator would have reported FAILED or IN_DOUBT rather than inventing a success.

9.6 How It Lives On in v2.3

In the prototype, negotiation priced a stand-alone scenario. In the product, `procurement.py` runs the same buyer–supplier negotiation once with each AI provider, scaled to that provider's list price, and caches the committed contract for an hour. The router then scores providers on the negotiated price instead of the list price, so the negotiation changes which model receives the prompt.

9.7 Submission Analysis

Table 9.4 maps the evidence in the submission to each rubric criterion.

Table 9.4: Rubric coverage for Assignment 3.1

Criterion	Evidence	Assessment
Multi-instance stability	Three agents run together; bounded rounds and sizes	Fully met
Negotiation protocol	CFP, proposal, counter-offers, acceptance rule with formulas	Fully met
Consensus handshake	PREPARE / READY / COMMIT / ACK over a SHA-256 digest	Fully met; 31 tests

- **Strengths.** The outcome is fully explained by two simple formulas, and failure is reported honestly.
- **Limits.** The prototype priced a stand-alone scenario; v1.0 wired it into provider procurement.

CHAPTER 10
ASSIGNMENT 3.2: SELF-INTERESTED UTILITY OPTIMIZATION

THE ASSIGNMENT IN ONE LINE
Given many options, pick the best one for yourself, using a clear scoring formula, without ever picking an option that breaks a hard rule.

10.1 What Was Asked

The final assignment asked for a self-interested agent that scores options with a utility function and picks the best one that obeys every hard rule (Table 10.1).

Table 10.1: Assignment 3.2 rubric

Criterion	Marks	What the examiner looks for
Utility equation	2	A defined, weighted formula with normalised terms
Optimization search	2	A correct and efficient search for the best option
Constraint resilience	1	Infeasible options are never chosen, with reasons
Total	**5**	

10.2 What Was Delivered

An `optimization` package (engine, models, scenarios and demo), a ranking CSV, a written report and 53 passing tests. The scenario: a buyer agent must purchase 100 units from ten e-commerce sellers, ECO-01 to ECO-10, each with a price, rating, delivery time, reliability and return rate.

10.3 The Utility Equation

The agent combines five scores, each from 0 to 100, with fixed weights (Table 10.2):

$$U = 0.30\,q + 0.25\,c + 0.20\,d + 0.15\,r + 0.10\,t$$

Table 10.2: Utility terms and how each is scored from 0 to 100

Term	Weight	Meaning	Score
q	0.30	Product quality (seller rating)	rating ÷ 5 × 100
c	0.25	Cost, within the $145 budget	(145 − landed cost) ÷ 145 × 100
d	0.20	Delivery speed	(5 − days) ÷ 4 × 100
r	0.15	Seller reliability	on-time percentage
t	0.10	Low return rate	(10 − return %) ÷ 10 × 100

For example, ECO-08 NorthPeak scores 0.30×94 + 0.25×9.66 + 0.20×100 + 0.15×97.5 + 0.10×60 = **71.24**.

10.4 Hard Constraints

Before any scoring matters, an offer must pass every hard rule (Table 10.3). An offer that fails is kept in the report with its reasons, but it can never be selected.

Table 10.3: Hard constraints in Assignment 3.2

Rule	Limit
Landed cost per unit	at most $145
Units in stock	at least 100
Seller rating	at least 4.0 of 5
Delivery time	at most 5 days
Reliability	at least 95%
Return rate	at most 6%
Seller status	active
Destination	served by the seller

10.5 The Search

Figure 10.1 shows the five steps the engine takes.

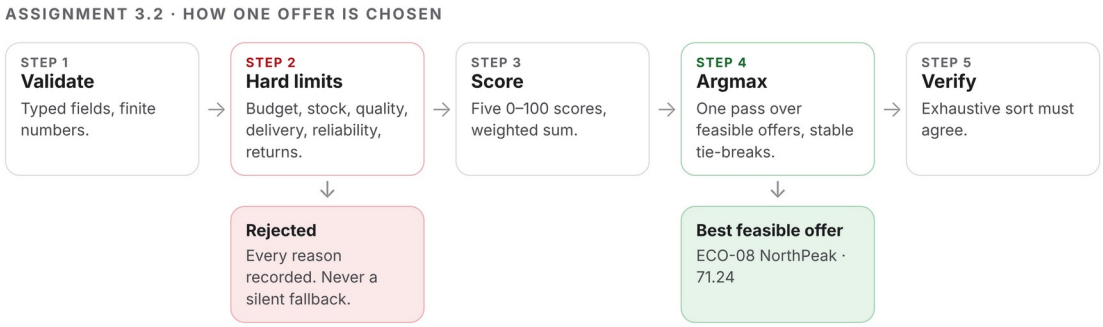


Figure 10.1: The five steps from offers to a single verified choice

The engine validates every field, applies the hard rules, scores each offer, and finds the best feasible one in **a single pass**, which takes time proportional to the number of offers, O(n). Ties are broken in a fixed order (higher utility, then lower cost, then faster delivery, then ID) so the result is always the same. As a safety net, an independent exhaustive sort is also computed and must agree with the single-pass winner.

10.6 Results

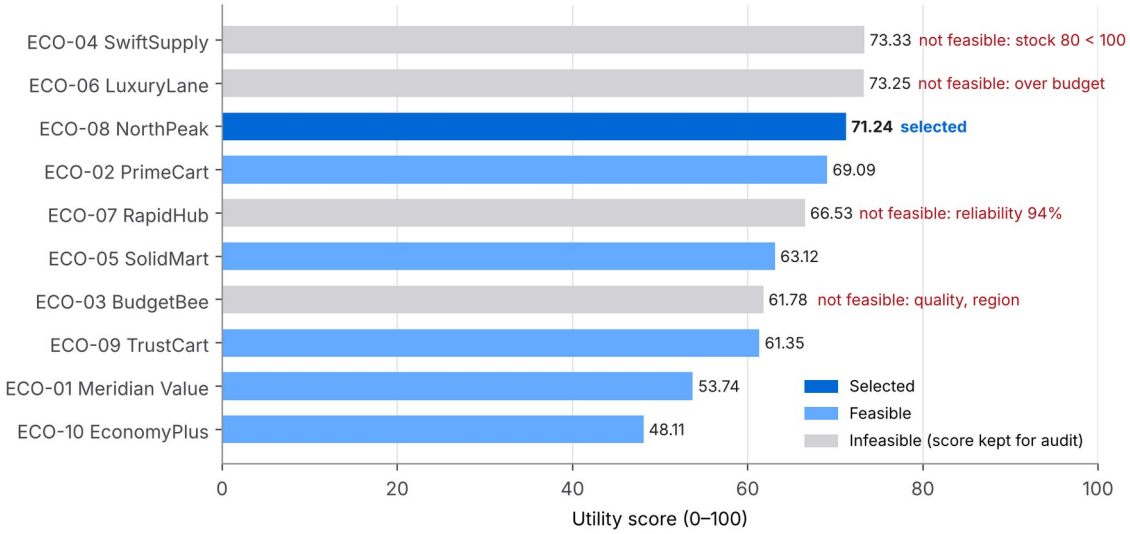


Figure 10.2: Utility of all ten offers; the selected offer is not the highest raw score

Figure 10.2 shows the key lesson of the assignment. The two highest scores, SwiftSupply (73.33) and LuxuryLane (73.25), are both **infeasible**: SwiftSupply has only 80 units in stock and LuxuryLane costs $148, over budget. RapidHub and BudgetBee also fail a rule. The best *feasible* offer is **ECO-08 NorthPeak, at a landed cost of $131.00 per unit and a utility of 71.24**. Six offers were feasible, ranked ECO-08 > ECO-02 > ECO-05 > ECO-09 > ECO-01 > ECO-10.

10.7 How It Lives On in v2.3

The router that chooses an AI provider is this optimizer applied to CipherMesh's own domain (Section 4.6). The weights became quality, cost, latency, availability and privacy fit, and the hard constraints became price, latency, quality, availability and data residency. In the live runs reported in Chapter 13, prompts with Indian personal data went to AWS Bedrock in Mumbai, while the others went to Vertex AI, which had the highest utility.

10.8 Submission Analysis

Table 10.4 maps the evidence in the submission to each rubric criterion.

Table 10.4: Rubric coverage for Assignment 3.2

Criterion	Evidence	Assessment
Utility equation	Five normalised terms with weights summing to 1	Fully met; worked example checks out
Optimization search	O(n) argmax, fixed tie-breaks, exhaustive oracle check	Fully met
Constraint resilience	Top two raw scores rejected with recorded reasons	Fully met; 53 tests

- **Strengths.** The example shows clearly why hard rules must come before scores.
- **Limits.** The weights are fixed by hand; learning them from user choices is future work.

CHAPTER 11
PRODUCT AND USER EXPERIENCE

11.1 Two Experiences, One Product

CipherMesh has two faces. The **product site** explains the platform to someone who has never seen it: what it does, why it matters, how it works and how to build with it. The **console** is where people actually check prompts and where reviewers make decisions. Both follow Apple's Human Interface Guidelines [38]: frosted glass only on floating controls such as the navigation bar and sidebar, quiet solid surfaces for content, and the device's own light or dark appearance. Figure 11.1 shows the landing page.

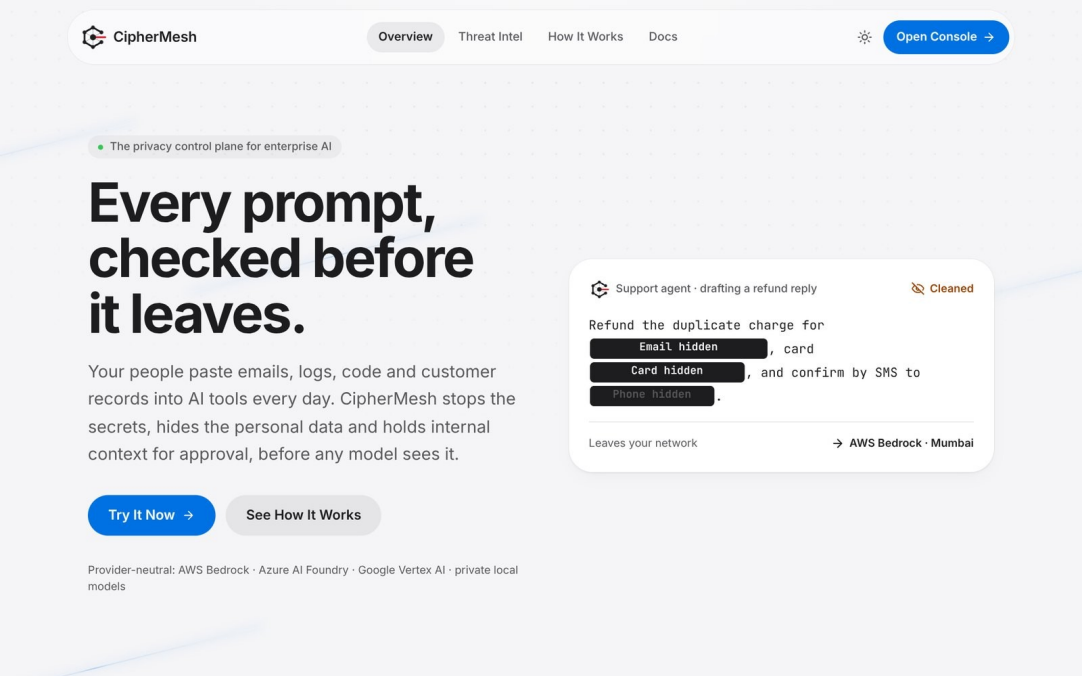


Figure 11.1: Product site landing page (light appearance)

11.2 The Product Site

The landing page states the promise in one line, "Every prompt, checked before it leaves", and shows a live example: a support agent's refund reply in which the email, card and phone number are covered by redaction bars. The **Threat Intel** page (Figure 11.2) charts 38 sourced AI security incidents by year and attack vector, and a live feed on the same page (not shown) lists the latest headlines from the AI Incident Database [34].



Figure 11.2: Threat intelligence dashboard: 38 sourced incidents by year and attack vector

The **How It Works** page explains the four layers and the agent core, and lets a visitor run a real check and watch each engineering module light up as the request passes through it (Figure 11.3). Each card names the course module it came from and the code that implements it.

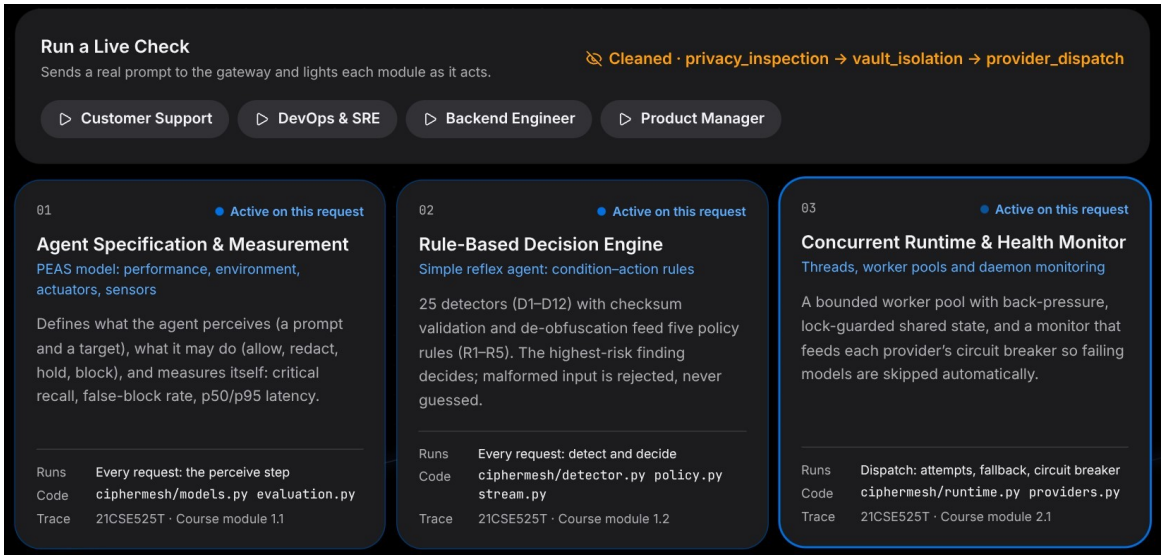


Figure 11.3: Engineering foundations lit up by a live check

11.3 The Console

The console opens on **Check**. A person pastes text, or picks one of ten real-world examples written for roles such as customer support, HR, DevOps and legal, and presses Check. The result arrives in plain words, with a matching colour and icon, so colour is never the only signal. Figures 11.4 to 11.7 show the four possible results.

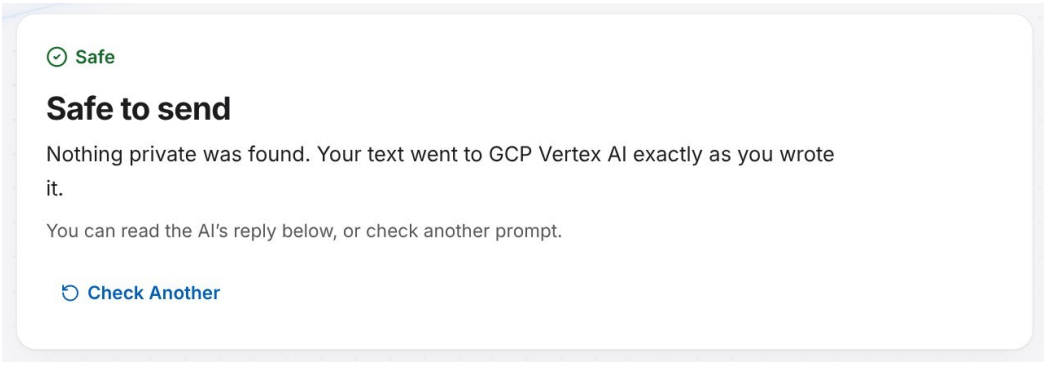


Figure 11.4: Safe: a product manager's PRD outline, sent exactly as written

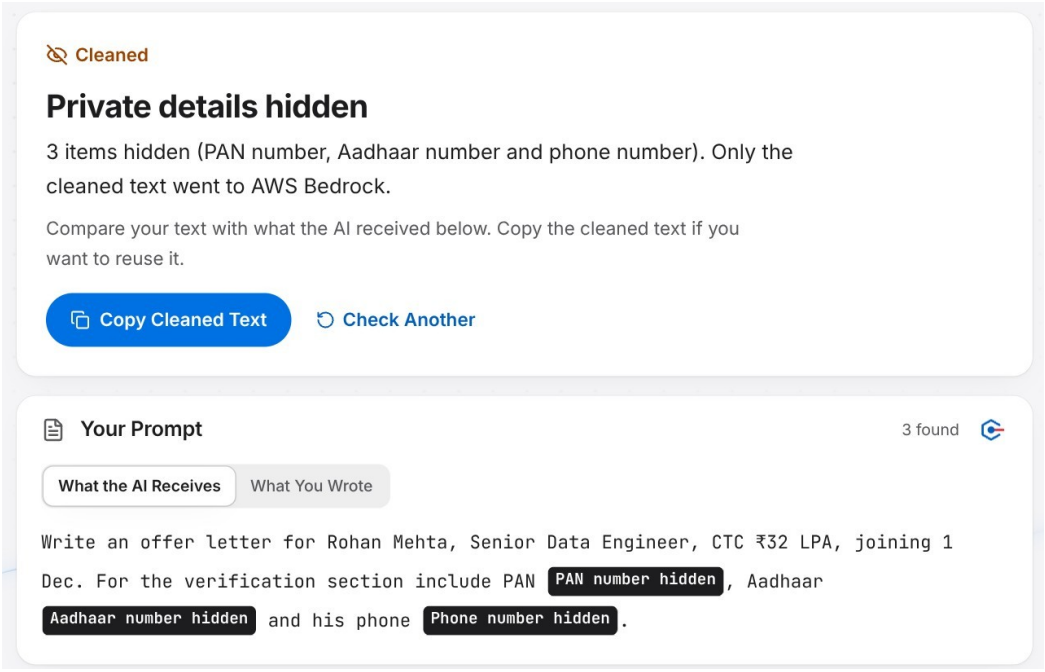


Figure 11.5: Cleaned: an offer letter with PAN, Aadhaar and phone number hidden before sending

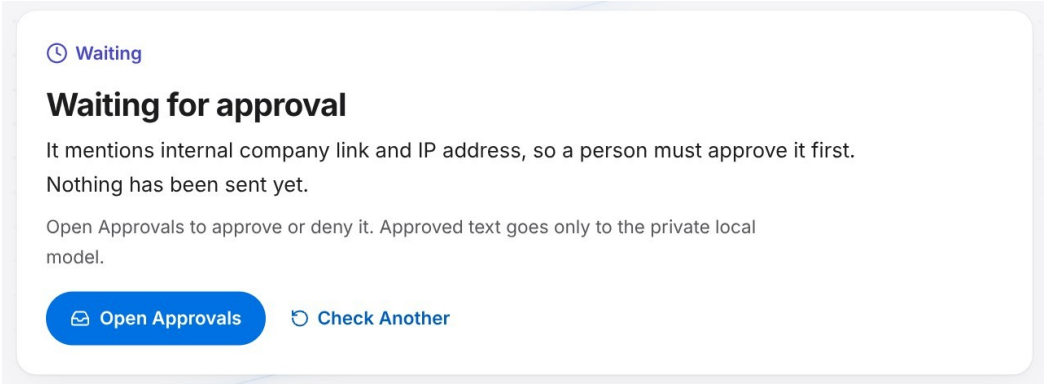


Figure 11.6: Waiting: an incident summary with internal links, held for a reviewer

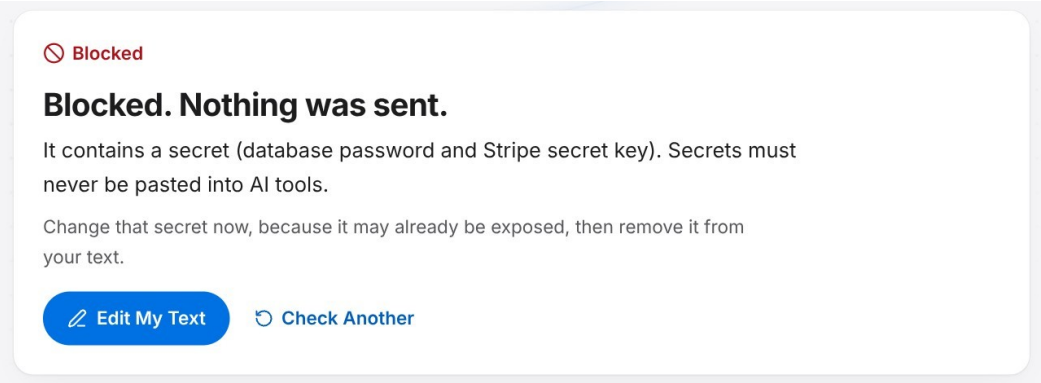


Figure 11.7: Blocked: a deploy log with a database password and a Stripe key; nothing was sent

The signature moment is the **redaction sweep**: after a check, each private value is shown for a beat, then a bar sweeps across exactly that value and labels what it hides, such as "PAN number hidden" (Figure 11.5). Below the result, the person can compare *What the AI Receives* with *What You Wrote*, read the model's reply, and open **Technical Details** to see the model choice table, the graph path, the vault record and the rules that fired.

Reviewers work in **Approvals**, where each held prompt is shown with its private parts masked, the time left before it expires, and Approve and Deny buttons that require the reviewer's name. Figure 11.8 shows the Technical Details panel in the dark appearance, with the model-choice table that the router produced.

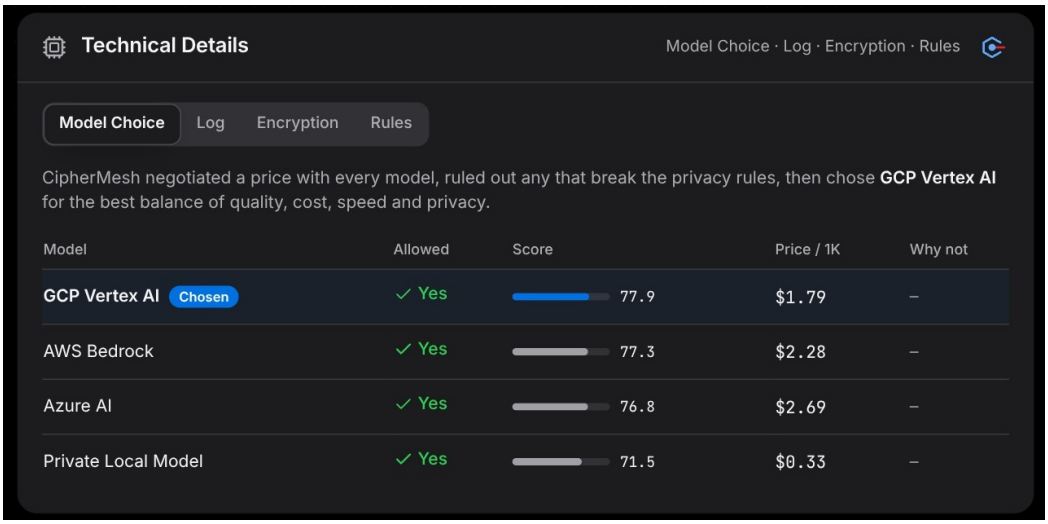


Figure 11.8: Technical Details: every provider scored with its negotiated price; here all four are allowed and GCP Vertex AI is chosen

11.4 Phones and Accessibility

On a phone the console uses an iOS-style tab bar, the site collapses into a menu, and sheets rise from the bottom with a grabber (Figure 11.9). Every screen was checked with axe-core [33] and a pixel-level contrast sampler that measured more than 15,000 text runs against the real rendered glass in both appearances; every run meets the WCAG 2.2 AA contrast ratio of 4.5:1 [39].

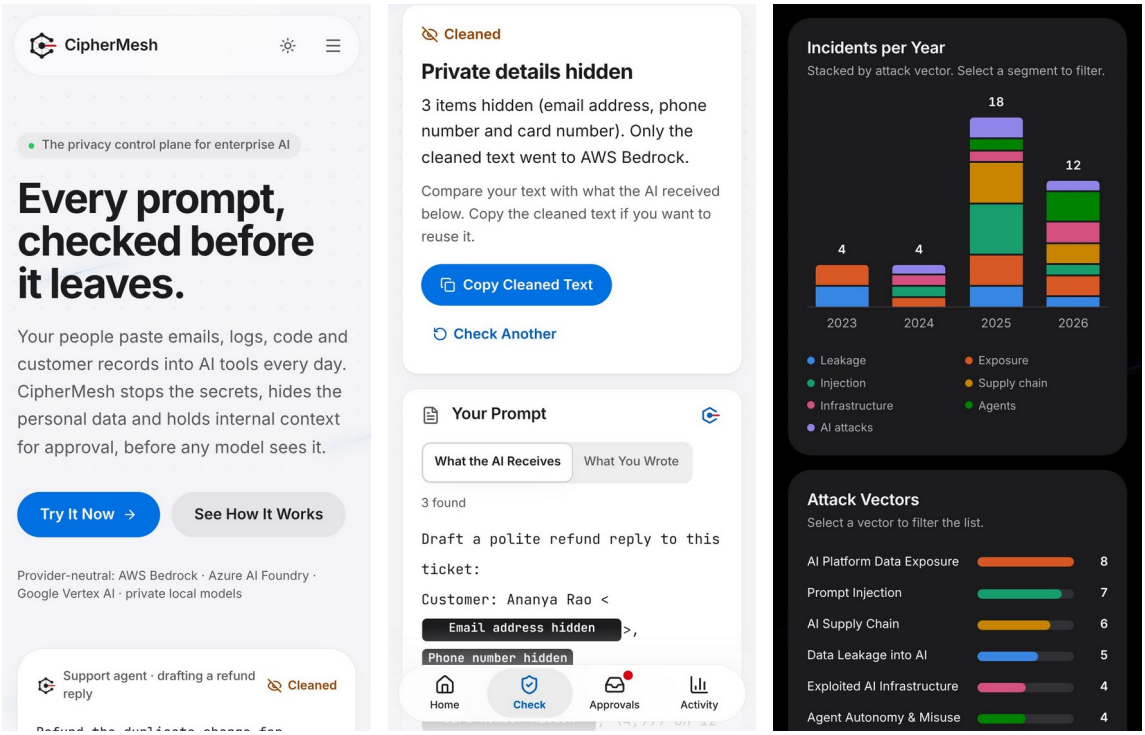


Figure 11.9: Product site, a cleaned result and the threat dashboard on a 390-pixel phone

CHAPTER 12
IMPLEMENTATION AND DEPLOYMENT

12.1 Repository Structure

The repository has one folder for the Python backend and one for the Next.js frontend:

```
CipherMesh/
├── backend/
│   ├── app/          FastAPI: main.py, api/, agents/graph.py, core/security.py
│   ├── ciphermesh/   engine: detector, policy, review, routing, runtime ...
│   ├── tests/        167 automated tests
│   └── Dockerfile, vercel.json, requirements(-dev).txt
├── frontend/
│   ├── app/          site: /, threats, architecture, docs
│   │                 console: check, approvals, activity
│   ├── lib/          api.ts, threats.ts (38), samples.ts (10)
│   └── Dockerfile
├── docs/             API contract, audit, brand, diagrams
├── docker-compose.yml
└── .github/workflows/ci.yml
```

12.2 Development History

Table 12.1 traces the project from the first prototype to the current product.

Table 12.1: From first prototype to product

Stage	Milestone
A1.1 and A1.2	Design specification and standard-library reflex agent
A2.1	Concurrent worker pool, monitor daemon and benchmark
A2.2 (v0.2.0)	Packaged gateway with socket messaging and 22 tests
A3.1 (v0.3.0)	Negotiation package with two-phase commit and 31 tests
A3.2 (v0.4.0)	Utility optimizer with hard constraints and 53 tests
v1.0.0	Unification: one engine, twelve audit findings fixed, HMAC-signed messages, review queue
v2.0.0 to v2.2.0	FastAPI + LangGraph + Next.js product; encrypted vault; glass interface and brand
v2.3.0	Product site, threat intelligence with live feed, documentation, role-based examples

12.3 Running Locally

The whole system starts with one command, `docker compose up --build`, after which the console is on port 3000 and the API on port 8000. Developers can also run each part directly: create a Python virtual environment, `pip install -r requirements-dev.txt` and `uvicorn app.main:app --reload` for the backend; `npm ci` and `npm run dev` for the frontend. The interactive API documentation is generated automatically at `/docs`.

12.4 Deployment

The live system runs on **Vercel** as two projects: the API from the `backend` folder and the console from the `frontend` folder. In production the backend refuses to start without a strong vault seed and salt, and the reviewer screens stay disabled unless a reviewer token is set. GitHub

Actions runs the backend tests, the TypeScript check, the linter and a production build on every push to `main` and on every pull request. Table 12.2 lists the settings a production deployment needs.

Table 12.2: Production settings

Variable	Purpose
`CIPHERMESH_ENV=production`	Turns on the strict start-up checks
`CIPHERMESH_VAULT_SEED`, `_SALT`	Master secret (32+ random characters recommended, 16 enforced) and per-deployment salt
`CIPHERMESH_REVIEW_TOKEN`	Reviewer password; without it, production disables the review queue
`CIPHERMESH_RATE_LIMIT_PER_MIN`	Requests per minute per client (default 120)
`NEXT_PUBLIC_API_BASE_URL`	Tells the console where the API lives

12.5 Security Hardening

Table 12.3 maps each threat to the protection built into the product.

Table 12.3: Protections built into the product

Threat	Protection
Secret sent to a provider	BLOCK sends nothing and returns no preview, not even a partial one
Evidence tampering or reading	Fernet-sealed records; audit log holds metadata only
Prompt fingerprinting	Keyed HMAC fingerprint in the agent graph; audit events keep only a short hash
Leaked data in replies	Model responses are inspected; leaked secrets are withheld
Large or chunked uploads	Streamed 400 KB body cap
Request floods	Sliding-window rate limit on every route
Browser attacks	CSP with frame-ancestors none, nosniff, X-Frame-Options, CORS without credentials; HSTS from the Vercel edge
Reflected input	Validation errors never echo the request

CHAPTER 13
TESTING AND RESULTS

13.1 Testing Strategy

Testing was layered. **Unit tests** check each rule and formula on its own; **integration tests** run whole prompts through the LangGraph agent and the API; and a **regression test** checks that overlapping findings never leave part of a secret visible. Separately from the automated suite, Playwright runs drove the real console in a headless browser. All 167 backend tests pass (Figure 13.1).

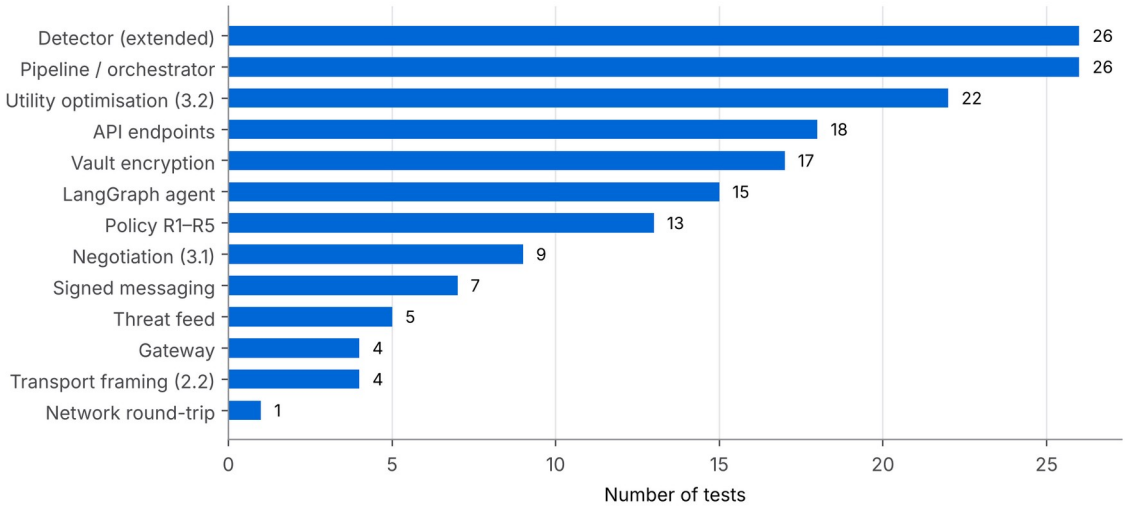


Figure 13.1: The 167 automated tests by suite, all passing

13.2 Functional Results on Real-World Examples

The ten role-based examples from the console were sent to the running API (Table 13.1). Every one produced the expected decision, and the router respected data residency: prompts with Indian personal data went to AWS Bedrock in Mumbai, and the other prompts that were sent went to Vertex AI, the highest-utility provider.

Table 13.1: Live results for the ten role-based examples

Role	What the prompt contained	Decision	Routed to
Customer Support	Email, Indian phone, card number	REDACT	AWS Bedrock (IN)
Backend Engineer	Database URL with password, Stripe key	BLOCK	Not sent
HR & Payroll	PAN, Aadhaar, Indian phone	REDACT	AWS Bedrock (IN)
DevOps & SRE	Internal URLs, IP address	REVIEW	Held
Product Manager	Nothing sensitive	ALLOW	GCP Vertex AI
Sales & RevOps	Emails, international phone	REDACT	GCP Vertex AI
Data Analyst	Database connection string	BLOCK	Not sent
Mobile Engineer	GitHub token	BLOCK	Not sent
Legal & Compliance	US SSN, email	REDACT	GCP Vertex AI
Security Engineer	JSON Web Token	BLOCK	Not sent

13.3 Detector Evaluation Against the PEAS Goals

The performance measures defined in Assignment 1.1 are computed by `evaluation.py` on a labelled corpus of 61 cases, split on purpose so that weaknesses are not hidden inside an average (Table 13.2).

Table 13.2: Detector evaluation by split

Split	Cases	Accuracy	Critical recall	False blocks	Leaks	p95 time
Known formats	40	100%	100%	0%	0	≈0.1 ms
Hard negatives	15	100%	n/a	0%	0	<0.1 ms
Adversarial	6	50%	50%	0%	2	≈0.2 ms
Overall	**61**	**95.1%**	**91.3%**	**0%**	**2**	**≈0.1 ms**

The headline PEAS target, **zero critical secrets forwarded for known formats, is met**, and no harmless text was wrongly blocked. The adversarial split shows the honest limit of a pattern-based detector: an access key broken into two pieces inside a line can slip through, and a phone number spelled out in words is not hidden. Hard negatives contain no secrets, so critical recall does not apply to them. Detection itself takes a fraction of a millisecond; most of a request's time is the provider call.

13.4 Performance

Assignment 2.1 showed near-linear scaling for I/O-bound work: 7.61× throughput at eight workers (Chapter 7). In the product, the agent's own work (inspection, vault sealing and routing) is small next to the model call, so the gateway adds little delay to each prompt.

13.5 Audit Findings and Fixes

When the six prototypes were joined in v1.0, an internal audit found twelve issues, mostly caused by each prototype having evolved its own copy of the detector. All were fixed and each has a test. The most important are listed in Table 13.3.

Table 13.3: Key audit findings fixed in v1.0

Severity	Finding	Fix
Critical	Some API tokens were allowed by the packaged gateway although the Assignment 1.2 agent blocked them	Dedicated detectors for each token family
High	Overlapping findings could leave part of a secret visible	Spans merged; the strictest category wins
High	REVIEW had no workflow	Review queue with TTL, re-inspection and local-only release
High	Message token proved knowledge but not integrity	HMAC-signed envelopes, freshness window, replay cache
Medium	Model replies were not inspected	Response-side inspection
Medium	Any long digit run was treated as a phone number	Proper phone patterns; Luhn and Verhoeff checks

13.6 Design and Accessibility Review

The interface was reviewed three ways: a functional pass through every screen, an automated accessibility scan with axe-core (no violations), and a design review against Apple's guidelines. The review led to concrete fixes, such as raising the dark-mode hover contrast, removing a nested interactive element, and stopping long code from breaking mid-word.

13.7 Limitations

- **Simulated cloud providers.** Bedrock, Azure and Vertex are realistic simulations so the demo costs nothing; prices and latencies are illustrative.
- **Serverless memory.** On Vercel the review queue lives in each running instance; a production deployment would back it with Redis or Postgres.
- **Reviewer identity.** The reviewer token proves permission, not who the person is; single sign-on is future work.
- **Obfuscation.** Secrets broken into pieces, or identifiers written out in words, are not always caught.
- **Internal hostnames.** A link such as `https://host.internal.example.com/path` is only partly masked: the rule stops at the internal label, so the public-looking suffix stays visible. Extending rule D8 to the full hostname is a planned fix.

CHAPTER 14
CONCLUSION AND FUTURE WORK

14.1 Conclusion

This project set out to let organisations use many AI models without leaking what matters. CipherMesh does that by putting an agent between applications and providers. The agent perceives every prompt, reasons with validated rules, and acts in one of four clear ways. It keeps sealed evidence instead of raw text, asks a person when a case is borderline, and negotiates and chooses a provider only among those that respect privacy rules.

Each assignment of the Agent Technology course contributed a real part of this behaviour: the PEAS design gave the goals and measures, the reflex reasoner gave the decision core, the concurrent runtime gave scale, messaging gave remote control, negotiation gave procurement and the utility optimizer gave routing. Joining them exposed twelve gaps, which were fixed and tested. The result is a working, deployed product with 167 passing tests, zero known-format secret leaks in evaluation, and an interface that meets accessibility standards.

14.2 Future Work

- **Smarter detection.** Add a small local language model to catch confidential text that has no fixed pattern, and to rejoin secrets broken into pieces.
- **Identity.** Single sign-on for reviewers, with a rule that the person who submitted a prompt cannot approve it.
- **Transport security.** Mutual TLS between agents and per-agent keys with rotation.
- **Durable state.** Back the review queue and metrics with Postgres or Redis for multi-instance deployments.
- **Real providers.** Connect live Bedrock, Azure and Vertex accounts, with measured latency feeding the router.
- **Policy as code.** Let each team write and test its own policy rules in the console.

REFERENCES

[1] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications 2025," OWASP GenAI Security Project, Nov. 2024. Accessed: Oct. 27, 2026. [Online]. Available: https://genai.owasp.org/llm-top-10/

[2] M. Gurman, "Samsung bans staff's AI use after spotting ChatGPT data leak," *Bloomberg*, May 2, 2023. Accessed: Oct. 27, 2026. [Online]. Available: https://www.bloomberg.com/news/articles/2023-05-02/samsung-bans-chatgpt-and-other-generative-ai-use-by-staff-after-leak

[3] Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023, *The Gazette of India*, Aug. 2023.

[4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[5] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.

[6] H. P. Luhn, "Computer for verifying numbers," U.S. Patent 2 950 048, Aug. 23, 1960.

[7] J. Verhoeff, *Error Detecting Decimal Codes* (Mathematical Centre Tracts, vol. 29). Amsterdam, The Netherlands: Mathematisch Centrum, 1969.

[8] N. Carlini *et al.*, "Extracting training data from large language models," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 2633–2650.

[9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[10] Amazon Web Services, "Detect and filter harmful content by using Amazon Bedrock Guardrails," *Amazon Bedrock User Guide*. Accessed: Oct. 27, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html

[11] Microsoft, "What is Azure AI Content Safety?" *Microsoft Learn*. Accessed: Oct. 27, 2026. [Online]. Available: https://learn.microsoft.com/azure/ai-services/content-safety/overview

[12] Microsoft, "Presidio: Data protection and de-identification SDK," GitHub repository. Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/microsoft/presidio

[13] GitHub, "About secret scanning," *GitHub Docs*. Accessed: Oct. 27, 2026. [Online]. Available: https://docs.github.com/code-security/secret-scanning

[14] Amazon Web Services, "Security best practices in IAM," *AWS Identity and Access Management User Guide*. Accessed: Oct. 27, 2026. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

[15] Google Cloud, "Best practices for managing service account keys," *Google Cloud IAM Documentation*. Accessed: Oct. 27, 2026. [Online]. Available: https://cloud.google.com/iam/docs/best-practices-for-managing-service-account-keys

[16] Microsoft, "Grant limited access to Azure Storage resources using shared access signatures (SAS)," *Microsoft Learn*. Accessed: Oct. 27, 2026. [Online]. Available: https://learn.microsoft.com/azure/storage/common/storage-sas-overview

[17] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," *IEEE Trans. Comput.*, vol. C-29, no. 12, pp. 1104–1113, Dec. 1980.

[18] P. Faratin, C. Sierra, and N. R. Jennings, "Negotiation decision functions for autonomous agents," *Robotics and Autonomous Systems*, vol. 24, no. 3–4, pp. 159–182, 1998.

[19] J. Gray, "Notes on data base operating systems," in *Operating Systems: An Advanced Course* (Lecture Notes in Computer Science, vol. 60). Berlin, Germany: Springer, 1978, pp. 393–481.

[20] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: Wiley, 2009.

[21] R. L. Keeney and H. Raiffa, *Decisions With Multiple Objectives: Preferences and Value Tradeoffs*. New York, NY, USA: Wiley, 1976.

[22] Python Cryptographic Authority, "Fernet (symmetric encryption)," *cryptography documentation*. Accessed: Oct. 27, 2026. [Online]. Available: https://cryptography.io/en/latest/fernet/

[23] K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: Password-based cryptography specification version 2.1," IETF RFC 8018, Jan. 2017.

[24] OWASP Foundation, "Password Storage Cheat Sheet," *OWASP Cheat Sheet Series*. Accessed: Oct. 27, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[25] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, Feb. 1997.

[26] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, vol. L 119, pp. 1–88, May 2016.

[27] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Jan. 2023.

[28] LangChain, Inc., "LangGraph documentation." Accessed: Oct. 27, 2026. [Online]. Available: https://langchain-ai.github.io/langgraph/

[29] S. Ramírez, "FastAPI documentation." Accessed: Oct. 27, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[30] Pydantic Services Inc., "Pydantic documentation (v2)." Accessed: Oct. 27, 2026. [Online]. Available: https://docs.pydantic.dev/

[31] SQLite Development Team, "Write-ahead logging," *SQLite Documentation*. Accessed: Oct. 27, 2026. [Online]. Available: https://www.sqlite.org/wal.html

[32] Vercel, Inc., "Next.js documentation." Accessed: Oct. 27, 2026. [Online]. Available: https://nextjs.org/docs

[33] Deque Systems, "axe-core: Accessibility engine for automated web UI testing," GitHub repository. Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/dequelabs/axe-core

[34] Responsible AI Collaborative, "AI Incident Database." Accessed: Oct. 27, 2026. [Online]. Available: https://incidentdatabase.ai/

[35] Ollama, "Ollama: Get up and running with large language models locally," GitHub repository. Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/ollama/ollama

[36] Python Software Foundation, "threading: Thread-based parallelism," *Python 3.12 Documentation*. Accessed: Oct. 27, 2026. [Online]. Available: https://docs.python.org/3.12/library/threading.html

[37] D. Beazley, "Understanding the Python GIL," presented at PyCon 2010, Atlanta, GA, USA, Feb. 2010.

[38] Apple Inc., "Human Interface Guidelines," *Apple Developer Documentation*, 2026. Accessed: Oct. 27, 2026. [Online]. Available: https://developer.apple.com/design/human-interface-guidelines

[39] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 2023.

APPENDIX A
API REFERENCE

Table A.1 lists every endpoint and Table A.2 the error codes. The base URL of the hosted API is `https://ciphermesh-api.vercel.app`.

Table A.1: REST endpoints

Method	Endpoint	What it does	Auth
POST	`/api/v1/inspect`	Runs the agent on a prompt	Optional API token
GET	`/api/v1/reviews?status=PENDING`	Lists held prompts	Reviewer token
POST	`/api/v1/reviews/{id}/decision`	Approves or denies a held prompt	Reviewer token
GET	`/api/v1/health`	Service status	None
GET	`/api/v1/metrics`	Decision counts and latency	None
GET	`/api/v1/providers`	Providers with negotiated prices	None
GET	`/api/v1/graph`	Live LangGraph definition (Mermaid)	None
GET	`/api/v1/threats/live`	Security-related AI incident headlines	None

Table A.2: Error codes

Code	Meaning
401	Missing or wrong token
403	Review console disabled on this deployment
404	Unknown or expired review
409	Review already decided
413	Body too large
422	Invalid input (the prompt is never echoed back)
429	Rate limited

A shortened example request and response:

```
POST /api/v1/inspect
{
  "prompt": "Email jane.doe@example.com and Aadhaar 2345 6789 0124",
  "target_provider": "auto"
}

{
  "action_decision": "REDACT",
  "status": "COMPLETED",
  "processed_prompt": "Email [REDACTED:EMAIL] and Aadhaar [REDACTED:AADHAAR]",
  "routed_provider": "aws-bedrock",
  "graph_path": ["privacy_inspection", "vault_isolation", "provider_dispatch"],
  "vault": { "kdf": "PBKDF2-HMAC-SHA256", "iterations": 600000 }
}
```

APPENDIX B
CONFIGURATION REFERENCE

Table B.1 lists the backend settings and their defaults. The frontend needs only `NEXT_PUBLIC_API_BASE_URL`.

Table B.1: Backend environment variables

Variable	Default	Purpose
`CIPHERMESH_ENV`	development	`production` enforces the secrets below
`CIPHERMESH_VAULT_SEED`	none	Vault master secret; required in production
`CIPHERMESH_VAULT_SALT`	none	Per-deployment salt; required in production
`CIPHERMESH_VAULT_ITERATIONS`	600,000	PBKDF2 iterations (minimum 100,000)
`CIPHERMESH_VAULT_PREVIOUS_SEEDS`	none	Old seeds kept readable during rotation
`CIPHERMESH_REVIEW_TOKEN`	none	Reviewer password
`CIPHERMESH_REVIEW_TTL_S`	900	Seconds a held prompt waits
`CIPHERMESH_API_TOKEN`	none	Optional token for /inspect
`CIPHERMESH_CORS_ORIGINS`	*	Allowed browser origins
`CIPHERMESH_RATE_LIMIT_PER_MIN`	120	Per-client limit; 0 disables
`CIPHERMESH_MAX_BODY_BYTES`	400,000	Largest accepted request
`CIPHERMESH_LATENCY_SCALE`	0.02	Speed of simulated providers
`CIPHERMESH_OLLAMA`	0	1 sends local traffic to a real Ollama model

APPENDIX C
USER GUIDE

1. Open the product site at `ciphermesh-console.vercel.app` and choose **Open Console**.
2. Paste the text you plan to send to an AI model, or tap one of the ten real-world examples.
3. Choose a model or leave **Best Match** to let CipherMesh pick, then press **Check** (Ctrl + Enter).
4. Read the result: **Safe**, **Cleaned**, **Waiting** or **Blocked**. For Cleaned text, use **Copy Cleaned Text**.
5. If the result is Blocked, change the exposed key or password first; it may already be compromised.
6. Reviewers open **Approvals**, read the masked text, enter their name and choose **Approve** or **Deny**. Approved text goes only to the private local model.
7. Open **Activity** to see totals for the gateway and the prompts you checked in this session.